

Rebuilding the Foundations of Logic: Sets, Type Theory, and Reproducibility

Developed from Pop, Refuse, Bind, and Collapse

Flyxion
Independent Researcher

October 2026

Abstract

Three foundational vocabularies dominate current practice: first-order logic with its Boolean semantics, Zermelo–Fraenkel set theory with choice (ZFC), and the dependent type theories implemented in proof assistants such as Lean. They are usually introduced separately, with separate primitives and separate informal pictures of what a mathematical object is. This monograph rebuilds all three from one small base. The base is four operators, *Pop* (introduce a distinguished item), *Refuse* (restrict a domain of admissible items), *Bind* (relate or abstract over items), and *Collapse* (evaluate or identify). With these operators the untyped lambda calculus is recovered by renaming, the classical and intuitionistic propositional logics appear as two disciplines of Refuse, the axioms of ZFC appear as closure conditions on the four operators, and the kernel of a Lean-style dependent type theory appears as the internalization of Refuse inside Bind. The development includes the missing floors: first-order logic with structures, satisfaction and soundness; simply typed lambda calculus with an explicit Curry–Howard theorem; a worked derivation in the kernel; and the diagonal arguments of Cantor, Russell, and Gödel, kept distinct. Reproducibility is treated as part of the foundations: each statement carries a status label, each verified theorem is reported with its axiom footprint, and the layers at which a formal statement can fail are classified, so that a reader can tell what was established, relative to what, and by what kind of check. The axioms of ZFC, and the axioms added to a kernel, are treated as conventions: arbitrary at the base, and valuable because they give everyone a common language in which results can be checked, combined, and reused. Each chapter separates what is defined, what is proved here, what is cited from the literature, and what is only an analogy. The aim is a single ground-up account in which the differences between the three foundations are located precisely, at the point where each decides how Refuse may be used.

Contents

1	Scope, conventions, and status vocabulary	5
1.1	What this document is	5
1.2	Status vocabulary	5
1.3	Scope and warrant	5
1.4	Content of a correspondence	5
1.5	Notation	6

I	Ground	6
2	Pops, worlds, and the four operators	6
2.1	An informal picture	6
2.2	The domain layer	6
2.3	Why four	7
3	The term layer: Bind and Collapse give the lambda calculus	7
3.1	Syntax and reduction	7
3.2	Worked reductions	8
3.3	What the term layer cannot say	8
II	Logic	8
4	Propositions as worlds: the Boolean discipline	8
4.1	Semantics by Refuse and Bind	8
4.2	Soundness and completeness	9
5	Two disciplines of Refuse	9
5.1	Topological worlds	9
5.2	The discipline as a design choice	10
6	Quantifiers, and the first dictionary	10
7	First-order logic, structures, and satisfaction	11
7.1	Syntax	11
7.2	Structures and satisfaction	11
7.3	Consequence and a proof system	12
III	Sets	13
8	The axioms of ZFC read as closure conditions	13
8.1	The theory	13
8.2	ZFC as a first-order theory	14
8.3	The empty set is not a separate primitive	14
8.4	Refuse needs a domain	14
8.5	Foundation and Bind-history	15
8.6	Choice as a section of Collapse	15
8.7	Models and the limits of the theory	16

9	Diagonal arguments	16
9.1	The schema	17
9.2	Four instances	17
9.3	What the schema does and does not say	17
9.4	Relation to Refuse	18
IV	Types	18
10	Simple types and the Curry–Howard correspondence	18
10.1	The calculus	18
10.2	Intuitionistic natural deduction	19
10.3	Normalization and consistency	19
10.4	What the dependent kernel adds	20
11	Refuse inside Bind: the kernel of a dependent type theory	21
11.1	What changes relative to the term layer	21
11.2	Judgments and the rules	21
11.3	Refuse as subtype formation	21
11.4	Impredicativity of Prop	22
11.5	Inductive types and the empty type	22
11.6	A worked derivation	23
11.7	Quotient types as Collapse	24
11.8	Sets inside types	25
12	Axioms added to the kernel	25
12.1	Three standard additions	25
12.2	Choice carries classical logic	25
12.3	The axiom footprint	26
13	Translating between sets and types	26
13.1	What is known	26
13.2	The second dictionary	27
13.3	Libraries and kernels	27
V	Reflection	27
14	Revising the admissibility rule	27
14.1	The operator A	27

15 Where a statement can fail	28
16 Conventions as a common language	29
16.1 The role of an arbitrary base	29
16.2 Reproducible results	29
17 Arbitrary bases, determinate consequences	30
17.1 Arbitrary at the base, forced above it	30
17.2 How much does the convention matter?	31
17.3 What a convention can be assessed on	31
18 One object, many representations: the radix example	31
18.1 Representation systems and canonical forms	31
18.2 Integer radices	32
18.3 A non-integer radix	32
18.4 Why integer radices are used	34
18.5 The representation schema	35
19 A reproducibility protocol	35
19.1 What a report must contain	35
19.2 Controls	36
19.3 What reproducibility does not give	37
20 A reading of the RSVP plenum	37
20.1 Purpose and status	37
20.2 The operator reading	37
20.3 What is lost by identification	37
20.4 An analogy and a conjecture	38
20.5 Implications if the conjecture held	38
20.6 What would settle it	39
21 Summary and open problems	39
21.1 What was built	39
21.2 Open problems	40
21.3 Exercises	40
A Inventory of cited results	41
B Summary of status labels by part	41
C Script for the radix computations	41

1 Scope, conventions, and status vocabulary

1.1 What this document is

The text develops a common base for logic, sets, and types, and then reads three standard systems off that base. It is not a new foundation intended to compete with them. Its claim is more modest: the three standard systems differ mainly in *where* they permit a domain to be restricted, and a four-operator vocabulary makes that difference easy to state.

1.2 Status vocabulary

Every numbered statement carries one of the following labels, and the label is part of the claim.

Label	Meaning
Definition	A stipulation. It can be adopted or declined but not refuted.
Theorem, Proposition, Lemma, Corollary	Proved in this text, or proved by a short argument given in full.
Cited result	Proved in the literature and used here without reproof. A reference is always given.
Computation	A finite calculation whose script is reproduced in an appendix and was run. It supports the stated instances only.
Design rule	A choice made for convenience or to match a standard system.
Analogy	A structural resemblance. It organizes understanding and proves nothing.
Conjecture	A statement supported by heuristic or partial evidence and not proved.

1.3 Scope and warrant

Two things are kept apart throughout. The *scope* of a result is the set of objects it is about: for example, a theorem about subsets of a fixed carrier is not a theorem about all collections. The *warrant* of a result is the kind of support it has: a proof, a citation, or an analogy. A theorem keeps the scope of its hypotheses; an analogy has no scope beyond the sentence that states it. When a reading of one system in terms of another is offered, the text says which of the two it is.

1.4 Content of a correspondence

Design rule 1.1 (Content of a correspondence). An operator correspondence has mathematical content only when its domain, its interpretation, and the structure it preserves are specified. Otherwise it is notation or analogy.

Every correspondence offered in this text is labeled accordingly: a Theorem or Proposition when the three are given and a preservation result is proved, and an Analogy when they are not. Section 18 gives the smallest complete example.

1.5 Notation

The carrier of all items is a set $\mathbb{U}$ (the *pops*). A *world* is a subset $\Omega \subseteq \mathbb{U}$. Predicates on $\mathbb{U}$ are identified with subsets. Substitution is written $t[x := u]$. The four operators are typeset **Pop**, **Refuse**, **Bind**, **Collapse**; a fifth symbol **A** is used in Section 14 for revision of the admissibility rule itself.

Part I

Ground

2 Pops, worlds, and the four operators

2.1 An informal picture

A mathematical activity begins by distinguishing something from its surroundings. That act is *Pop*. It continues by deciding which items count as admissible for the question at hand, and by excluding the rest. That act is *Refuse*. Admissible items are then related to each other, composed, paired, or abstracted over: *Bind*. Finally, related items are evaluated or identified, so that different presentations of the same thing become one: *Collapse*. The rest of the document makes these four sentences precise.

2.2 The domain layer

Definition 2.1 (Worlds and operators). Fix a set $\mathbb{U}$. For $a \in \mathbb{U}$, $P \subseteq \mathbb{U}$, worlds $\Omega, \Omega_1, \Omega_2 \subseteq \mathbb{U}$, a relation $R \subseteq \mathbb{U} \times \mathbb{U}$, and an equivalence relation $\sim$ on Ω , define

$$\begin{aligned} \text{Pop}(a) &:= \{a\}, \\ \text{Refuse}_P(\Omega) &:= \Omega \setminus P, \\ \text{Bind}_R(\Omega_1, \Omega_2) &:= R \cap (\Omega_1 \times \Omega_2), \quad \text{Bind}(\Omega_1, \Omega_2) := \Omega_1 \times \Omega_2, \\ \text{Collapse}_{\sim}(\Omega) &:= \Omega / \sim. \end{aligned}$$

The operators have different codomains: **Pop**, **Refuse** return worlds, **Bind** returns a world of pairs (a world over $\mathbb{U} \times \mathbb{U}$, which is again a carrier of items after a choice of pairing), and **Collapse** returns a world of classes. Pairing and classes are items of a suitable $\mathbb{U}$ in all the systems considered below, so no generality is lost by keeping the carrier fixed.

Proposition 2.2 (Laws of Refuse). *For all $P, Q \subseteq \mathbb{U}$ and worlds Ω :*

- (i) $\text{Refuse}_P \text{Refuse}_Q(\Omega) = \text{Refuse}_{P \cup Q}(\Omega) = \text{Refuse}_Q \text{Refuse}_P(\Omega);$
- (ii) $\text{Refuse}_P \text{Refuse}_P(\Omega) = \text{Refuse}_P(\Omega);$
- (iii) $\text{Refuse}_{\emptyset}(\Omega) = \Omega$ and $\text{Refuse}_{\mathbb{U}}(\Omega) = \emptyset;$
- (iv) (relative double refusal) $\text{Refuse}_{\text{Refuse}_P(\Omega)}(\Omega) = \Omega \cap P.$

Proof. (i) $x \in \text{Refuse}_P \text{Refuse}_Q(\Omega)$ iff $x \in \Omega$, $x \notin Q$, $x \notin P$ iff $x \in \Omega \setminus (P \cup Q)$. Symmetry in P, Q gives the second equality. (ii) is (i) with $Q = P$. (iii) is immediate. (iv) $x \in \Omega \setminus (\Omega \setminus P)$ iff $x \in \Omega$ and not $(x \in \Omega \text{ and } x \notin P)$ iff $x \in \Omega \cap P$. $\square$

Item (iv) is the first place where the rest of the document will branch. In the Boolean setting it says that refusing a refusal, relative to a fixed world, returns what one started with. In Section 5 the same operator is shown to fail this law in a topological setting, and the failure is exactly the failure of excluded middle.

Proposition 2.3 (Collapse is the universal identification). *Let $\sim$ be an equivalence relation on a world Ω and $q : \Omega \rightarrow \Omega/\sim$ the quotient map. For every function $f : \Omega \rightarrow X$ that is constant on $\sim$ -classes there is a unique $\bar{f} : \Omega/\sim \rightarrow X$ with $\bar{f} \circ q = f$.*

Proof. Define $\bar{f}([x]) := f(x)$. This is well defined because f is constant on classes. Uniqueness holds because q is surjective. $\square$

Proposition 2.3 is the whole content of “Collapse as evaluation”: a computation that yields the same result on equivalent presentations descends to the identified object. The same universal property reappears in Section 11.7 as the rule for quotient types.

2.3 Why four

Analogy 2.4. The four operators correspond, in order, to *introducing a variable or constant, forming a subtype or subset, forming a function space or relation, and reducing a term or forming a quotient*. The correspondence is not claimed to be forced. Any presentation of logic needs some way of introducing items, restricting their range, relating them, and identifying them, and these are the four ways chosen here.

3 The term layer: Bind and Collapse give the lambda calculus

3.1 Syntax and reduction

Definition 3.1 (SP terms). Let X be a countable set of variables. The set Tm of *SP terms* is generated by

$$t, u ::= x \mid \text{Bind } x. t \mid \text{Collapse}(t, u), \quad x \in X.$$

Variables are pops introduced by the context. The term $\text{Bind } x. t$ binds the pop x in t , and $\text{Collapse}(t, u)$ asks that t be evaluated against u . Free variables, α -equivalence, and capture-avoiding substitution $t[x := u]$ are defined as for the lambda calculus. The single reduction rule is

$$\text{Collapse}((\text{Bind } x. t), u) \longrightarrow t[x := u] \quad (\text{Collapse-}\beta)$$

closed under all term contexts. Write $\rightarrow$ for its reflexive transitive closure.

Proposition 3.2 (The SP calculus is the lambda calculus). *The map $\tau : \text{Tm} \rightarrow \Lambda$ into the untyped lambda terms, $\tau(x) = x$, $\tau(\text{Bind } x. t) = \lambda x. \tau(t)$, $\tau(\text{Collapse}(t, u)) = \tau(t) \tau(u)$, is a bijection that preserves and reflects one-step reduction. In particular, $\rightarrow$ on Tm is confluent (Church–Rosser) and every theorem about β -reduction in Λ holds for Collapse- β .*

Proof. τ is a bijection of grammars, clause by clause, and commutes with substitution by induction on t . Hence $t \rightarrow u$ iff $\tau(t) \rightarrow_{\beta} \tau(u)$. Confluence of $\rightarrow_{\beta}$ is the Church–Rosser theorem, proved for example in Barendregt [4, Ch. 3]. $\square$

This proposition is deliberately a change of vocabulary. No new computational power or new theorem about computation is claimed. The value of the vocabulary appears in the next parts, where `Bind` and `Collapse` are re-used for quantification, for pairing, and for dependent products.

3.2 Worked reductions

The following reductions were also checked with a short normal-order evaluator; each is short enough to verify by hand. Write $T := \text{Bind } x. \text{Bind } y. x$, $F := \text{Bind } x. \text{Bind } y. y$, $\text{not} := \text{Bind } b. \text{Collapse}(\text{Collapse}(b, F), T)$, and $\text{and} := \text{Bind } p. \text{Bind } q. \text{Collapse}(\text{Collapse}(p, q), p)$.

$$\begin{aligned} \text{Collapse}(\text{not}, T) &\rightarrow \text{Collapse}(\text{Collapse}(T, F), T) \rightarrow F, \\ \text{Collapse}(\text{Collapse}(\text{and}, T), F) &\rightarrow \text{Collapse}(\text{Collapse}(T, F), T) \rightarrow F. \end{aligned}$$

With Church numerals $\bar{n} := \text{Bind } f. \text{Bind } x. f^n x$ and

$$\begin{aligned} \text{plus} &:= \text{Bind } m. \text{Bind } n. \text{Bind } f. \text{Bind } x. \text{Collapse}(\text{Collapse}(m, f), \text{Collapse}(\text{Collapse}(n, f), x)), \\ \text{mult} &:= \text{Bind } m. \text{Bind } n. \text{Bind } f. \text{Collapse}(m, \text{Collapse}(n, f)), \end{aligned}$$

one obtains $\text{plus } \bar{2} \bar{2} \rightarrow \bar{4}$ and $\text{mult } \bar{2} \bar{3} \rightarrow \bar{6}$. In every case the final form is reached by repeated use of the single rule.

3.3 What the term layer cannot say

The calculus above has no way to say that a term is *not* admissible in a given position. Every term can be applied to every term. This is a real limitation: it is why the untyped calculus has terms such as $\text{Collapse}((\text{Bind } x. \text{Collapse}(x, x)), (\text{Bind } x. \text{Collapse}(x, x)))$ that never reach a normal form. Restricting the range of a binder is the job of `Refuse`, and Part IV shows that doing so inside `Bind` yields dependent types.

Part II

Logic

4 Propositions as worlds: the Boolean discipline

4.1 Semantics by Refuse and Bind

Fix a nonempty world Ω_0 , the *ambient world* of a discussion. A proposition is interpreted as the sub-world of Ω_0 at which it holds.

Definition 4.1 (Boolean world semantics). Let $\mathcal{L}$ be the propositional language built from atoms, $\perp$, $\wedge$, $\rightarrow$, with $\neg\varphi := \varphi \rightarrow \perp$, $\varphi \vee \psi := \neg(\neg\varphi \wedge \neg\psi)$. A *valuation* v assigns to each atom a subset of Ω_0 . Extend v to $\llbracket \cdot \rrbracket$ by

$$\begin{aligned} \llbracket \perp \rrbracket &= \emptyset, \\ \llbracket \varphi \wedge \psi \rrbracket &= \llbracket \varphi \rrbracket \cap \llbracket \psi \rrbracket, \\ \llbracket \neg\varphi \rrbracket &= \text{Refuse}_{\llbracket \varphi \rrbracket}(\Omega_0), \\ \llbracket \varphi \rightarrow \psi \rrbracket &= \text{Refuse}_{\llbracket \varphi \rrbracket}(\Omega_0) \cup \llbracket \psi \rrbracket. \end{aligned}$$

The clause for $\wedge$ is Bind with the identity relation: $\text{Bind}_=(\Omega_1, \Omega_2) = \{(a, a) : a \in \Omega_1 \cap \Omega_2\}$ is in bijection with $\Omega_1 \cap \Omega_2$, so joint admissibility is binding two constraints to the same item. The clause for $\neg$ is Refuse applied to the ambient world.

Proposition 4.2. *For every valuation, $\llbracket \neg\neg\varphi \rrbracket = \llbracket \varphi \rrbracket$, $\llbracket \varphi \vee \neg\varphi \rrbracket = \Omega_0$, and $\llbracket \neg(\varphi \wedge \psi) \rrbracket = \llbracket \neg\varphi \vee \neg\psi \rrbracket$.*

Proof. By Proposition 2.2(iv), $\llbracket \neg\neg\varphi \rrbracket = \text{Refuse}_{\text{Refuse}_{\llbracket \varphi \rrbracket}(\Omega_0)}(\Omega_0) = \Omega_0 \cap \llbracket \varphi \rrbracket = \llbracket \varphi \rrbracket$, since $\llbracket \varphi \rrbracket \subseteq \Omega_0$. For excluded middle, $\llbracket \varphi \vee \neg\varphi \rrbracket = \Omega_0 \setminus ((\Omega_0 \setminus \llbracket \varphi \rrbracket) \cap (\Omega_0 \setminus (\Omega_0 \setminus \llbracket \varphi \rrbracket))) = \Omega_0 \setminus ((\Omega_0 \setminus \llbracket \varphi \rrbracket) \cap \llbracket \varphi \rrbracket) = \Omega_0$. The third identity is the set-theoretic De Morgan law relative to Ω_0 . $\square$

4.2 Soundness and completeness

Natural deduction for classical propositional logic has the rules: assumption; $\wedge$ -introduction and elimination; $\rightarrow$ -introduction (discharge) and elimination (modus ponens); $\perp$ -elimination; and reductio (from $\Gamma, \neg\varphi \vdash \perp$ infer $\Gamma \vdash \varphi$). Write $\Gamma \models \varphi$ if, for every valuation, $\bigcap_{\gamma \in \Gamma} \llbracket \gamma \rrbracket \subseteq \llbracket \varphi \rrbracket$ (with the empty intersection equal to Ω_0).

Theorem 4.3 (Soundness). *If $\Gamma \vdash \varphi$ then $\Gamma \models \varphi$.*

Proof. Induction on derivations. Write $G = \bigcap_{\gamma \in \Gamma} \llbracket \gamma \rrbracket$. *Assumption:* $G \subseteq \llbracket \gamma \rrbracket$ for $\gamma \in \Gamma$. $\wedge I$, $\wedge E$: immediate from $\llbracket \varphi \wedge \psi \rrbracket = \llbracket \varphi \rrbracket \cap \llbracket \psi \rrbracket$. $\rightarrow I$: suppose $G \cap \llbracket \varphi \rrbracket \subseteq \llbracket \psi \rrbracket$ and let $x \in G$. If $x \in \llbracket \varphi \rrbracket$ then $x \in \llbracket \psi \rrbracket$; otherwise $x \in \Omega_0 \setminus \llbracket \varphi \rrbracket$. Either way $x \in \llbracket \varphi \rightarrow \psi \rrbracket$. $\rightarrow E$: if $x \in G$ lies in $(\Omega_0 \setminus \llbracket \varphi \rrbracket) \cup \llbracket \psi \rrbracket$ and in $\llbracket \varphi \rrbracket$ then $x \in \llbracket \psi \rrbracket$. $\perp E$: if $G \subseteq \emptyset$ then $G \subseteq \llbracket \varphi \rrbracket$. *Reductio*: if $G \cap \llbracket \neg\varphi \rrbracket \subseteq \emptyset$ then $G \subseteq \Omega_0 \setminus \llbracket \neg\varphi \rrbracket = \llbracket \neg\neg\varphi \rrbracket = \llbracket \varphi \rrbracket$ by Proposition 4.2. $\square$

Cited result 4.4 (Completeness). If $\Gamma \models \varphi$ in the two-element ambient world $\Omega_0 = \{0, 1\}$ (that is, by truth tables), then $\Gamma \vdash \varphi$. See Troelstra and van Dalen [18, Ch. 2].

Taken together, Theorem 4.3 and Cited result 4.4 say that classical propositional logic is the logic of Refuse and $\text{Bind}_=$ over worlds when Refuse is involutive relative to the ambient world.

5 Two disciplines of Refuse

5.1 Topological worlds

The previous section used a bare set as the ambient world. If the ambient world carries a topology and admissible regions are required to be open, the same four-clause semantics produces a different logic.

Definition 5.1 (Open-world semantics). Let X be a topological space. Propositions are interpreted as open subsets of X , with $\llbracket \perp \rrbracket = \emptyset$, $\llbracket \varphi \wedge \psi \rrbracket = \llbracket \varphi \rrbracket \cap \llbracket \psi \rrbracket$, $\llbracket \varphi \vee \psi \rrbracket = \llbracket \varphi \rrbracket \cup \llbracket \psi \rrbracket$, and

$$\llbracket \neg\varphi \rrbracket = \text{int}(X \setminus \llbracket \varphi \rrbracket), \quad \llbracket \varphi \rightarrow \psi \rrbracket = \text{int}((X \setminus \llbracket \varphi \rrbracket) \cup \llbracket \psi \rrbracket).$$

Refuse is still “remove the region and keep what remains,” but what remains must again be an admissible (open) region, so the interior is taken.

Cited result 5.2 (Soundness for intuitionistic logic). Every intuitionistically derivable sequent is valid in all open-world semantics. Conversely, a propositional formula valid in all such semantics is intuitionistically derivable. See Troelstra and van Dalen [18, Ch. 2].

Lemma 5.3 (Refuse as largest disjoint admissible region). *For open $U \subseteq X$, $\neg U := \text{int}(X \setminus U)$ is the largest open set disjoint from U .*

Proof. An open set W is disjoint from U iff $W \subseteq X \setminus U$ iff (since W is open) $W \subseteq \text{int}(X \setminus U)$. And $\text{int}(X \setminus U)$ is itself open and disjoint from U . $\square$

Proposition 5.4 (What survives). *For open U, V : (a) $U \subseteq \neg\neg U$; (b) $U \subseteq V$ implies $\neg V \subseteq \neg U$; (c) $\neg\neg\neg U = \neg U$.*

Proof. (a) U is open and disjoint from $\neg U$, so by Lemma 5.3 applied to $\neg U$, $U \subseteq \neg\neg U$. (b) $\neg V$ is open and disjoint from $V \supseteq U$, hence disjoint from U , so $\neg V \subseteq \neg U$ by the lemma. (c) Apply (a) to $\neg U$ to obtain $\neg U \subseteq \neg\neg\neg U$. Apply (b) to the inclusion in (a) to obtain $\neg\neg\neg U \subseteq \neg U$. $\square$

Example 5.5 (Double refusal expands, and excluded middle fails). Let $X = \mathbb{R}$ with its usual topology and $U = \mathbb{R} \setminus \{0\}$, which is open. Then $\neg U = \text{int}(\{0\}) = \emptyset$, so $\neg\neg U = \text{int}(\mathbb{R}) = \mathbb{R} \neq U$, and $\llbracket U \vee \neg U \rrbracket = U \cup \emptyset = U \neq \mathbb{R}$. Hence in this semantics neither $\neg\neg\varphi \rightarrow \varphi$ nor $\varphi \vee \neg\varphi$ is valid. By Cited result 5.2, neither is intuitionistically derivable.

5.2 The discipline as a design choice

Design rule 5.6 (Which Refuse). *Classical discipline:* admissible regions are arbitrary subsets of the ambient world, and Refuse is the relative complement. Then $\text{RefuseRefuse} = \text{id}$ (Proposition 2.2(iv)). *Constructive discipline:* admissible regions must be stable under some notion of refinement (open sets, in the example), and Refuse is the largest admissible region disjoint from the refused one. Then RefuseRefuse may strictly enlarge, and only Proposition 5.4 survives.

Analogy 5.7. In the open-world example, $\neg\neg U \setminus U = \{0\}$ is the part of the world that cannot be distinguished from U by any open refinement but is not in U . The constructive refusal declines to decide about such items. This is a picture of why proof assistants that are constructive by default treat excluded middle as an additional input rather than a theorem. It proves nothing beyond Example 5.5.

6 Quantifiers, and the first dictionary

For a world D and a family of propositions $\varphi(d)$, $d \in D$, interpret

$$\llbracket \forall x \in D. \varphi(x) \rrbracket = \bigcap_{d \in D} \llbracket \varphi(d) \rrbracket, \quad \llbracket \exists x \in D. \varphi(x) \rrbracket = \bigcup_{d \in D} \llbracket \varphi(d) \rrbracket,$$

in the Boolean setting; in the open-world setting the universal clause takes the interior of the intersection. The first is a Bind over D (a family indexed by D , collected jointly), the second is the union of a Bind-family. Soundness for the usual quantifier rules is the standard argument and is not repeated here.

Analogy 6.1 (The first dictionary). The following correspondence is the Curry–Howard reading specialised to the four operators. The row headings are the operators; the columns are the three vocabularies treated in this text.

Operator	Logic	Sets	Types
Pop	atom, hypothesis	element, urelement-free atom*	variable, constructor
Refuse	negation, restriction	Separation	subtype, $\neg A := A \rightarrow \perp$
Bind	$\wedge, \vee, \rightarrow$	pairing, products, functions	Π, Σ, λ
Collapse	modus ponens, evaluation	quotient, Replacement	application, β , Quot

* Pure ZFC has no atoms; pure sets are built from the empty set by the operators. The entry records the role, not an ontological commitment.

7 First-order logic, structures, and satisfaction

ZFC is a theory in first-order logic. Section 6 gave the quantifier clauses informally. This section supplies the missing floor: a precise syntax, structures, satisfaction, consequence, a proof system, and the soundness theorem, with completeness cited.

7.1 Syntax

Definition 7.1 (Signature, terms, formulas). A *signature* Σ is a set of function symbols and relation symbols, each with an arity (constants are 0-ary function symbols). Let Var be a countable set of variables. *Terms* are generated by $t ::= x \mid f(t_1, \dots, t_n)$ for $x \in \text{Var}$ and $f \in \Sigma$ of arity n . *Formulas* are generated by

$$\varphi, \psi ::= R(t_1, \dots, t_n) \mid t_1 = t_2 \mid \perp \mid \varphi \rightarrow \psi \mid \forall x \varphi.$$

The symbols $\neg, \wedge, \vee, \leftrightarrow$, and $\exists$ are abbreviations: $\neg\varphi := \varphi \rightarrow \perp$, $\exists x \varphi := \neg\forall x \neg\varphi$, and so on. The set $\text{FV}(\varphi)$ of free variables is defined as usual, with $\forall x$ binding x . The substitution $\varphi[t/x]$ replaces the free occurrences of x by t , and we write that t is free for x in φ when no variable of t is captured by a quantifier of φ at an occurrence being replaced.

The language of set theory has one binary relation symbol $\in$ and no function symbols; equality is part of the logic.

7.2 Structures and satisfaction

Definition 7.2 (Structure, assignment, satisfaction). A Σ -*structure* $\mathcal{M} = (D, (f^{\mathcal{M}}), (R^{\mathcal{M}}))$ consists of a nonempty set D , a function $f^{\mathcal{M}} : D^n \rightarrow D$ for each n -ary function symbol, and a relation $R^{\mathcal{M}} \subseteq D^n$ for each n -ary relation symbol. An *assignment* is a map $s : \text{Var} \rightarrow D$; write $s[x \mapsto d]$ for the assignment that agrees with s except that it sends x to d . The

value $t^{\mathcal{M},s}$ of a term is defined by $x^{\mathcal{M},s} = s(x)$ and $f(t_1, \dots, t_n)^{\mathcal{M},s} = f^{\mathcal{M}}(t_1^{\mathcal{M},s}, \dots, t_n^{\mathcal{M},s})$. *Satisfaction* $\mathcal{M}, s \models \varphi$ is defined by recursion on φ :

$$\begin{aligned} \mathcal{M}, s \models R(t_1, \dots, t_n) &\iff (t_1^{\mathcal{M},s}, \dots, t_n^{\mathcal{M},s}) \in R^{\mathcal{M}}, \\ \mathcal{M}, s \models t_1 = t_2 &\iff t_1^{\mathcal{M},s} = t_2^{\mathcal{M},s}, \\ \mathcal{M}, s \not\models \perp, \quad \mathcal{M}, s \models \varphi \rightarrow \psi &\iff \mathcal{M}, s \not\models \varphi \text{ or } \mathcal{M}, s \models \psi, \\ \mathcal{M}, s \models \forall x \varphi &\iff \text{for all } d \in D, \mathcal{M}, s[x \mapsto d] \models \varphi. \end{aligned}$$

The first-order semantics is the semantics of Section 4 with a larger ambient world. Let $\Omega_{\mathcal{M}} := D^{\text{Var}}$ be the world of assignments and $\llbracket \varphi \rrbracket_{\mathcal{M}} := \{s : \mathcal{M}, s \models \varphi\} \subseteq \Omega_{\mathcal{M}}$.

Proposition 7.3 (First-order semantics as worlds). *In $\Omega_{\mathcal{M}}$: $\llbracket \perp \rrbracket = \emptyset$; $\llbracket \neg \varphi \rrbracket = \text{Refuse}_{\llbracket \varphi \rrbracket}(\Omega_{\mathcal{M}})$; $\llbracket \varphi \wedge \psi \rrbracket = \llbracket \varphi \rrbracket \cap \llbracket \psi \rrbracket$; and*

$$\llbracket \forall x \varphi \rrbracket = \{s : s[x \mapsto d] \in \llbracket \varphi \rrbracket \text{ for every } d \in D\}, \quad \llbracket \exists x \varphi \rrbracket = \Omega_{\mathcal{M}} \setminus \llbracket \forall x \neg \varphi \rrbracket.$$

Consequently Proposition 4.2 holds in $\Omega_{\mathcal{M}}$: $\llbracket \neg \neg \varphi \rrbracket = \llbracket \varphi \rrbracket$ and $\llbracket \varphi \vee \neg \varphi \rrbracket = \Omega_{\mathcal{M}}$.

Proof. The propositional clauses are those of Definition 4.1 with $\Omega_0 = \Omega_{\mathcal{M}}$. The clause for $\forall$ is the definition of satisfaction. For $\exists$: $s \in \llbracket \exists x \varphi \rrbracket$ iff $s \notin \llbracket \forall x \neg \varphi \rrbracket$ iff there is d with $s[x \mapsto d] \notin \llbracket \neg \varphi \rrbracket$, that is with $s[x \mapsto d] \in \llbracket \varphi \rrbracket$. $\square$

In the vocabulary of the earlier parts, $\forall x$ is a **Bind** over the domain D (it collects the conditions at all values of x), and negation is **Refuse** in the ambient world of assignments.

Lemma 7.4 (Coincidence). *If s and s' agree on $\text{FV}(\varphi)$ then $\mathcal{M}, s \models \varphi \iff \mathcal{M}, s' \models \varphi$.*

Proof. Induction on φ . Terms have the same value under assignments that agree on their variables, which handles atomic formulas. The cases $\perp$ and $\rightarrow$ are immediate. For $\forall y \psi$: the assignments $s[y \mapsto d]$ and $s'[y \mapsto d]$ agree on $\text{FV}(\psi) \subseteq \text{FV}(\forall y \psi) \cup \{y\}$, so the induction hypothesis applies for every d . $\square$

Lemma 7.5 (Substitution). *If t is free for x in φ , then $\mathcal{M}, s \models \varphi[t/x] \iff \mathcal{M}, s[x \mapsto t^{\mathcal{M},s}] \models \varphi$.*

Proof. Induction on φ , after the corresponding statement for terms, $(u[t/x])^{\mathcal{M},s} = u^{\mathcal{M},s[x \mapsto t^{\mathcal{M},s}]}$, which is proved by induction on u . The only nontrivial case is $\varphi = \forall y \psi$. If $x \notin \text{FV}(\varphi)$ the substitution does nothing and the claim is Lemma 7.4. Otherwise $y \neq x$, and since t is free for x in φ , $y \notin \text{FV}(t)$. Then $\mathcal{M}, s \models (\forall y \psi)[t/x]$ iff for all d , $\mathcal{M}, s[y \mapsto d] \models \psi[t/x]$, iff (induction hypothesis, and $t^{\mathcal{M},s[y \mapsto d]} = t^{\mathcal{M},s}$ because $y \notin \text{FV}(t)$) for all d , $\mathcal{M}, s[y \mapsto d][x \mapsto t^{\mathcal{M},s}] \models \psi$, iff (the two updates commute as $x \neq y$) $\mathcal{M}, s[x \mapsto t^{\mathcal{M},s}] \models \forall y \psi$. $\square$

7.3 Consequence and a proof system

Definition 7.6 (Consequence). For a set Γ of formulas, $\Gamma \models \varphi$ means: for every structure $\mathcal{M}$ and assignment s with $\mathcal{M}, s \models \gamma$ for all $\gamma \in \Gamma$, also $\mathcal{M}, s \models \varphi$. A formula is *valid* if $\emptyset \models \varphi$. A *theory* is a set of sentences (formulas without free variables); $\mathcal{M}$ is a *model* of T if $\mathcal{M}, s \models \tau$ for every $\tau \in T$ (for sentences the choice of s is immaterial by Lemma 7.4).

Definition 7.7 (Natural deduction for first-order logic with equality). Sequents $\Gamma \vdash \varphi$ are derived by:

$$\begin{array}{c}
\frac{\varphi \in \Gamma}{\Gamma \vdash \varphi} \quad \frac{\Gamma, \varphi \vdash \psi}{\Gamma \vdash \varphi \rightarrow \psi} \quad \frac{\Gamma \vdash \varphi \rightarrow \psi \quad \Gamma \vdash \varphi}{\Gamma \vdash \psi} \quad \frac{\Gamma, \neg\varphi \vdash \perp}{\Gamma \vdash \varphi} \text{ (RAA)} \\
\\
\frac{\Gamma \vdash \varphi \quad x \notin \text{FV}(\Gamma)}{\Gamma \vdash \forall x \varphi} \text{ (\forall I)} \quad \frac{\Gamma \vdash \forall x \varphi \quad t \text{ free for } x}{\Gamma \vdash \varphi[t/x]} \text{ (\forall E)} \\
\\
\frac{}{\Gamma \vdash t = t} \text{ (= I)} \quad \frac{\Gamma \vdash t = u \quad \Gamma \vdash \varphi[t/x] \quad t, u \text{ free for } x}{\Gamma \vdash \varphi[u/x]} \text{ (= E)}
\end{array}$$

Theorem 7.8 (Soundness). *If $\Gamma \vdash \varphi$ then $\Gamma \models \varphi$.*

Proof. Induction on derivations. Let $\mathcal{M}, s$ satisfy Γ . The assumption, $\rightarrow$ I, $\rightarrow$ E, and RAA rules are verified as in Theorem 4.3, with satisfaction in place of membership in $\llbracket \cdot \rrbracket$ (the clauses for $\perp$ and $\rightarrow$ are the same, and RAA uses $\llbracket \neg\neg\varphi \rrbracket = \llbracket \varphi \rrbracket$ from Proposition 7.3). $\forall$ I. By the induction hypothesis $\Gamma \models \varphi$. Let $d \in D$. Since $x \notin \text{FV}(\Gamma)$, Lemma 7.4 gives $\mathcal{M}, s[x \mapsto d] \models \gamma$ for each $\gamma \in \Gamma$, so $\mathcal{M}, s[x \mapsto d] \models \varphi$. As d was arbitrary, $\mathcal{M}, s \models \forall x \varphi$. $\forall$ E. By the induction hypothesis $\mathcal{M}, s \models \forall x \varphi$, so $\mathcal{M}, s[x \mapsto d] \models \varphi$ with $d = t^{\mathcal{M}, s}$. By Lemma 7.5, $\mathcal{M}, s \models \varphi[t/x]$. =I. $t^{\mathcal{M}, s} = t^{\mathcal{M}, s}$. =E. By hypothesis $t^{\mathcal{M}, s} = u^{\mathcal{M}, s}$ and $\mathcal{M}, s \models \varphi[t/x]$, i.e. (Lemma 7.5) $\mathcal{M}, s[x \mapsto t^{\mathcal{M}, s}] \models \varphi$. Since $t^{\mathcal{M}, s} = u^{\mathcal{M}, s}$ this is $\mathcal{M}, s[x \mapsto u^{\mathcal{M}, s}] \models \varphi$, hence $\mathcal{M}, s \models \varphi[u/x]$ by Lemma 7.5 again. $\square$

Corollary 7.9 (A model gives consistency). *If a theory T has a model, then $T \not\vdash \perp$.*

Proof. If $T \vdash \perp$ then, by Theorem 7.8, $\mathcal{M}, s \models \perp$ for any model $\mathcal{M}$ of T , which is impossible by Definition 7.2. $\square$

Cited result 7.10 (Completeness, compactness). (a) If $\Gamma \models \varphi$ then $\Gamma \vdash \varphi$ (Gödel [23]; see also [10, Ch. 2]). (b) Consequently, a theory is consistent if and only if it has a model, and a theory has a model if each of its finite subsets does (compactness).

Part III

Sets

8 The axioms of ZFC read as closure conditions

8.1 The theory

The language of set theory has one binary predicate $\in$ and equality. The axioms of ZFC are the following sentences and schemas [13, 14]; here $x \subseteq y$ abbreviates $\forall z(z \in x \rightarrow z \in y)$.

Axiom	Statement	Operator reading
Extensionality	$\forall x \forall y (\forall z (z \in x \leftrightarrow z \in y) \rightarrow x = y)$	Collapse : same members, same item
Pairing	$\forall a \forall b \exists c \forall x (x \in c \leftrightarrow x = a \vee x = b)$	Bind of two pops
Union	$\forall F \exists A \forall x (x \in A \leftrightarrow \exists Y (x \in Y \wedge Y \in F))$	Bind flattened
Separation (schema)	$\forall a \exists b \forall x (x \in b \leftrightarrow x \in a \wedge \varphi(x))$	Refuse with a prior domain a
Replacement (schema)	if φ defines a function on a , its image is a set	Collapse of a definable function
Power set	$\forall a \exists b \forall x (x \in b \leftrightarrow x \subseteq a)$	all Refuse -subworlds of a
Infinity	$\exists a (\emptyset \in a \wedge \forall x \in a (x \cup \{x\} \in a))$	closure under successor Bind
Foundation	$\forall a (a \neq \emptyset \rightarrow \exists x \in a (x \cap a = \emptyset))$	well-founded Bind -history
Choice	every family of nonempty pairwise disjoint sets has a selector	section of a Collapse quotient

The right-hand column is an *Analogy* in the sense of Section 1. The theorems that follow are statements about ZFC itself.

8.2 ZFC as a first-order theory

Definition 8.1 (ZFC, formally). The language is $\Sigma_{\in} = \{\in\}$ with one binary relation symbol. ZFC is the theory consisting of the sentences of the table above, with each schema (Separation, Replacement) standing for the set of its instances, one for each formula φ and choice of parameters. A *model of ZFC* is a $\Sigma_{\in}$ -structure $(D, \in^{\mathcal{M}})$ satisfying all of them.

8.3 The empty set is not a separate primitive

Proposition 8.2 (Existence of $\emptyset$). *ZFC without any axiom asserting that a particular set exists proves $\exists e \forall x (x \notin e)$, and e is unique.*

Proof. First-order logic proves $\exists a (a = a)$, since the domain of a structure is nonempty. Apply Separation to this a with $\varphi(x) := x \neq x$ to obtain b with $x \in b \leftrightarrow x \in a \wedge x \neq x$. No x satisfies $x \neq x$, so b has no members. Uniqueness is Extensionality. $\square$

Proposition 8.2 is a statement about the standard first-order presentation. In the operator reading, $\emptyset = \text{Refuse}_{\mathbb{U}}(\Omega)$ for any world Ω : refusing everything leaves nothing, and this requires only that some world exist to refuse from.

8.4 Refuse needs a domain

Theorem 8.3 (No universal set). *ZF proves that for every set a there is a set r with $r \notin a$. In particular there is no set containing all sets.*

Proof. Given a , Separation yields $r := \{x \in a : x \notin x\}$. Suppose $r \in a$. If $r \in r$ then by definition of r , $r \notin r$; if $r \notin r$ then, since $r \in a$, $r \in r$. Either case is contradictory, so $r \notin a$. If a set V contained all sets, applying this to $a = V$ would give a set $r \notin V$, a contradiction. $\square$

The naive comprehension schema $\exists b \forall x (x \in b \leftrightarrow \varphi(x))$ is **Refuse** without a domain: it forms the region of everything satisfying φ from no prior world. Theorem 8.3 shows that, together

with the ordinary rules of logic, this is inconsistent for $\varphi(x) := x \notin x$. Separation repairs it by requiring $\text{Refuse}_P(\Omega)$ to be applied to an already given world $\Omega = a$.

Design rule 8.4 (Domain-first). In any system in which sets are collected from predicates, the predicate must be applied to an already admitted world. This is the form of the restriction in ZFC (Separation), and it recurs in type theory as the requirement that a subtype $\{x : A \mid p(x)\}$ be formed over an existing type A .

8.5 Foundation and Bind-history

Proposition 8.5 (No membership loops). *ZF proves: (i) $\forall x (x \notin x)$; (ii) there are no x, y with $x \in y$ and $y \in x$.*

Proof. (i) Apply Foundation to $\{x\}$, which is nonempty by Pairing. Its only element is x , so $x \cap \{x\} = \emptyset$. If $x \in x$ then $x \in x \cap \{x\}$, a contradiction. (ii) Apply Foundation to $\{x, y\}$. Either $x \cap \{x, y\} = \emptyset$ or $y \cap \{x, y\} = \emptyset$. But $y \in x \cap \{x, y\}$ and $x \in y \cap \{x, y\}$, a contradiction. $\square$

Under the operator reading, the pairing of items by **Bind** records a history: $x \in y$ says that x was bound before y . Foundation states that the history has a first element in every nonempty collection. The standard consequence is that every set has an ordinal rank.

Definition 8.6 (Cumulative hierarchy). $V_0 = \emptyset$; $V_{\alpha+1} = \mathcal{P}(V_\alpha)$; $V_\lambda = \bigcup_{\alpha < \lambda} V_\alpha$ for limit λ .

Cited result 8.7 (Rank). In ZF, Foundation is equivalent to the statement that every set belongs to some V_α . See Jech [13, Ch. 6].

Proposition 8.8 (Power set as all Refuse outputs). *For every world $\Omega \subseteq \mathbb{U}$, $\mathcal{P}(\Omega) = \{\text{Refuse}_P(\Omega) : P \subseteq \mathbb{U}\}$.*

Proof. $\text{Refuse}_P(\Omega) = \Omega \setminus P \subseteq \Omega$. Conversely, for $S \subseteq \Omega$ take $P = \Omega \setminus S$; then $\text{Refuse}_P(\Omega) = S$. $\square$

This yields a clean distinction between two axioms that are often conflated. *Separation* applies Refuse with a predicate P that is *definable* by a formula with parameters. *Power set* asserts that the collection of outputs of Refuse over *all* $P \subseteq \mathbb{U}$, definable or not, is itself a set. The difference between them is exactly the difference between a countable supply of refusals (formulas) and the full supply, and it is the source of the independence phenomena discussed below.

8.6 Choice as a section of Collapse

Proposition 8.9 (Choice and quotients). *In ZF the following are equivalent: (a) the Axiom of Choice; (b) for every surjection $q : \Omega \rightarrow Q$ there is $s : Q \rightarrow \Omega$ with $q \circ s = \text{id}_Q$.*

Proof. (a) $\Rightarrow$ (b): the sets $q^{-1}(c)$, $c \in Q$, are nonempty and pairwise disjoint; a selector for this family defines s . (b) $\Rightarrow$ (a): given a family F of nonempty sets, let $\Omega = \{(A, x) : A \in F, x \in A\}$ and $q(A, x) = A$. A section s gives $A \mapsto$ second component of $s(A)$, a choice function. $\square$

When q is the quotient map of Proposition 2.3, (b) says that every Collapse can be undone by a uniform choice of representative. In ZF without Choice this is not available in general, which is the content of the independence results below.

Cited result 8.10 (Independence of Choice). If ZF is consistent then so are ZFC (Gödel [11]) and $\text{ZF} + \neg\text{AC}$ (Cohen [6]). Hence AC is neither provable nor refutable in ZF.

Theorem 8.11 (Choice implies excluded middle). *Let p be any proposition. In an extensional set theory with Separation, Pairing, and the Axiom of Choice for families of nonempty finite sets, $p \vee \neg p$ holds, even if the underlying logic is intuitionistic.*

Proof. Let $A = \{x \in \{0, 1\} : x = 0 \vee p\}$ and $B = \{x \in \{0, 1\} : x = 1 \vee p\}$. Then $0 \in A$ and $1 \in B$, so both are nonempty, and by Choice there is f with $f(A) \in A$ and $f(B) \in B$. Hence $(f(A) = 0 \vee p)$ and $(f(B) = 1 \vee p)$. Distribute: either p holds, or $f(A) = 0$ and $f(B) = 1$. In the second case $f(A) \neq f(B)$. Now suppose p . Then $A = B = \{0, 1\}$ by Extensionality, so $f(A) = f(B)$. Therefore p implies $f(A) = f(B)$, and so in the second case $\neg p$. In summary, $p \vee \neg p$. $\square$

Theorem 8.11 is due to Diaconescu [9] and, in a form for sets, Goodman and Myhill [12]. It explains why Choice is never a mere convenience in a constructive setting: it carries the whole of classical logic with it. The same argument reappears in dependent type theory in Section 12.

8.7 Models and the limits of the theory

Cited result 8.12 (Models of ZFC). (a) If ZFC has a model it has a countable model (Löwenheim–Skolem). (b) If κ is a (strongly) inaccessible cardinal then V_κ is a model of ZFC. (c) Consequently, if ZFC is consistent then ZFC does not prove that an inaccessible cardinal exists. See Jech [13, Ch. 12 and Ch. 13] and Kunen [14, Ch. IV].

In the formal sense of Definition 8.1, “ZFC proves σ ” means $\text{ZFC} \vdash \sigma$ in the system of Definition 7.7, and Cited result 8.12(b) says that for κ inaccessible the structure $(V_\kappa, \in)$ is a model of ZFC. Corollary 7.9 gives the elementary half of the link between models and consistency used in Section 14.

The relevance for the next part is that each additional level of the universe hierarchy in a type theory will correspond to an additional inaccessible in the set-theoretic reading. “Larger universe” is an extra admissibility rule, not a consequence of the earlier ones.

9 Diagonal arguments

Four results in this text turn on a diagonal construction: Russell’s argument (Theorem 8.3), Cantor’s theorem, Gödel’s second theorem (Cited result 9.1 below), and the inconsistency of a type of all types. They share a form and differ in what they conclude. This section states the shared form precisely and then separates the instances.

Cited result 9.1 (Gödel’s second incompleteness theorem). If T is a consistent, recursively axiomatized theory that interprets elementary arithmetic, then $T \not\vdash \text{Con}(T)$.

9.1 The schema

Proposition 9.2 (Diagonal schema). *Let A and B be sets, $\nu : B \rightarrow B$ a function with no fixed point, and $f : A \rightarrow (A \rightarrow B)$ any function. Define $g : A \rightarrow B$ by $g(x) := \nu(f(x)(x))$. Then $g \neq f(a)$ for every $a \in A$. In particular f is not surjective onto $A \rightarrow B$.*

Proof. For $a \in A$, $g(a) = \nu(f(a)(a)) \neq f(a)(a)$ because ν has no fixed point, so g and $f(a)$ differ at a . $\square$

Theorem 9.3 (Cantor). *For every set A there is no surjection $f : A \rightarrow \mathcal{P}(A)$. In particular $\mathcal{P}(A)$ is not in bijection with A .*

Proof. Identify subsets with characteristic functions $A \rightarrow \{0, 1\}$ and take ν to swap 0 and 1. By Proposition 9.2, the set $D := \{x \in A : x \notin f(x)\}$ is not $f(a)$ for any a . (In the language of Section 2, $D = \text{Refuse}_P(A)$ with $P = \{x \in A : x \in f(x)\}$, so D is formed by Separation over the given set A .) $\square$

9.2 Four instances

Instance	Hypothesis used	What the diagonal produces	Conclusion
Cantor	A any set	a subset of A outside the range of f	a cardinality fact, no contradiction
Russell	A is a set containing all sets	the set $\{x \in A : x \notin x\}$	the hypothesis fails: there is no universal set (Theorem 8.3)
Gödel II	T consistent, recursively axiomatized, interprets arithmetic	a sentence equivalent to its own unprovability, built by the fixed-point lemma	$T \not\vdash \text{Con}(T)$ (Cited result 9.1)
Type of types	$\text{Type} : \text{Type}$	a paradox in the style of Girard and Hurkens	the rule is inconsistent; universe levels are needed

Cited result 9.4 (Fixed-point lemma and the type-of-types paradox). (a) For a theory T in the language of arithmetic and a formula $\psi(x)$ with one free variable, there is a sentence θ such that $T \vdash \theta \leftrightarrow \psi(\ulcorner \theta \urcorner)$, where $\ulcorner \theta \urcorner$ is the numeral of the Gödel number of θ [21]. (b) A type theory with a type of all types, $\text{Type} : \text{Type}$, is inconsistent (Girard’s paradox; see Hurkens [25] for a simplification).

9.3 What the schema does and does not say

The four results are not four forms of one theorem. Proposition 9.2 is the common construction. The instances differ in what is assumed (a set with a surjection in Cantor’s case, a universal set in Russell’s, an arithmetized provability predicate in Gödel’s, a universe containing itself in the type-theoretic case), and in what is concluded. Cantor’s argument refutes nothing: it shows that a size comparison has a definite answer. Russell’s argument refutes a hypothesis. Gödel’s argument requires a good deal more machinery: syntax must be represented inside arithmetic and provability must be representable, which is why the fixed-point lemma in Cited result 9.4(a) is needed and cannot be replaced by Proposition 9.2 alone. A categorical formulation of the common pattern is due to Lawvere [26]; it is cited here for orientation and not used.

Design rule 9.5 (Naming the diagonal). When a diagonal argument is invoked, state which hypothesis is being used and which kind of conclusion is drawn: a size fact, the refutation of a hypothesis, or a limit on provability. The schema does not by itself say which.

9.4 Relation to Refuse

Two of the instances bear on the central distinction of this text. The Russell argument fails to produce a set only when the domain of the restriction is not itself a set (Design rule 8.4). Cantor’s theorem shows that, over any domain A , the supply of Refuse outputs $\mathcal{P}(A)$ strictly exceeds the supply of elements, which is why the Power set axiom is a substantive axiom beyond Separation, and why a countable model cannot contain *all* of the refusals available externally (Remark 9.6). In type theory the type-of-types paradox plays the role of Russell’s: an unrestricted formation rule is inconsistent, and the universe hierarchy of Section 11 is the restriction that blocks it.

Remark 9.6 (Skolem’s observation in operator terms). By Cited results 8.12(a) and 7.10, if ZFC is consistent it has a countable model $\mathcal{M}$. Inside $\mathcal{M}$ the Power set axiom holds, so $\mathcal{M}$ contains an element which $\mathcal{M}$ regards as the set of all subsets of ω and, by Cantor’s theorem (Theorem 9.3), as uncountable. From outside, D is countable, so this set has countably many members. There is no contradiction: Power set asserts closure under the Refuse-outputs *that exist in $\mathcal{M}$* (Proposition 8.8), not under all of them. A first-order theory cannot fix the full supply of refusals; this is the content of the contrast between Separation and Power set drawn in Section 8.

Part IV

Types

10 Simple types and the Curry–Howard correspondence

The term layer of Section 3 has no types. A type discipline restricts which terms may be combined, and the restriction turns out to be exactly the restriction imposed by a proof system. This section makes the correspondence a theorem for the propositional fragment. The dependent type theory of Section 11 then generalizes it.

10.1 The calculus

Definition 10.1 (Types and terms of $\lambda^{\rightarrow \times + 0}$). Fix a set of base types $\alpha, \beta, \dots$. *Types* are $A, B ::= \alpha \mid A \rightarrow B \mid A \times B \mid A + B \mid \mathbf{0}$. *Terms* are

$t, u ::= x \mid \lambda x:A. t \mid t u \mid \langle t, u \rangle \mid \pi_1 t \mid \pi_2 t \mid \text{inl } t \mid \text{inr } t \mid \text{case } t \text{ of } \text{inl } x \Rightarrow u \mid \text{inr } y \Rightarrow v \mid \text{abort}_C t$.

The typing judgment $\Gamma \vdash t : A$ is generated by

$$\begin{array}{c} \frac{x:A \in \Gamma}{\Gamma \vdash x : A} \quad \frac{\Gamma, x:A \vdash t : B}{\Gamma \vdash \lambda x:A. t : A \rightarrow B} \quad \frac{\Gamma \vdash f : A \rightarrow B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B} \\[10pt] \frac{\Gamma \vdash a : A \quad \Gamma \vdash b : B}{\Gamma \vdash \langle a, b \rangle : A \times B} \quad \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \pi_1 p : A} \quad \frac{\Gamma \vdash p : A \times B}{\Gamma \vdash \pi_2 p : B} \end{array}$$

$$\begin{array}{c}
\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{inl}a : A + B} \quad \frac{\Gamma \vdash b : B}{\Gamma \vdash \text{inr}b : A + B} \quad \frac{\Gamma \vdash s : A + B \quad \Gamma, x:A \vdash u : C \quad \Gamma, y:B \vdash v : C}{\Gamma \vdash \text{case } s \text{ of } \text{inl}x \Rightarrow u \mid \text{inr}y \Rightarrow v : C} \\
\frac{\Gamma \vdash e : \mathbf{0}}{\Gamma \vdash \text{abort}_C e : C}
\end{array}$$

The reduction rules are

$$\begin{array}{l}
(\lambda x:A. t) a \rightarrow t[x := a], \quad \pi_i \langle a_1, a_2 \rangle \rightarrow a_i, \\
\text{case inl}a \text{ of } \text{inl}x \Rightarrow u \mid \text{inr}y \Rightarrow v \rightarrow u[x := a], \quad \text{case inr}b \text{ of } \text{inl}x \Rightarrow u \mid \text{inr}y \Rightarrow v \rightarrow v[y := b],
\end{array}$$

closed under term contexts.

In the vocabulary of the earlier parts, abstraction is **Bind**, application together with the β rule is **Collapse**, and the type $A \rightarrow \mathbf{0}$ is the **Refuse** of A : a way of discharging every inhabitant of A into the empty type.

10.2 Intuitionistic natural deduction

Definition 10.2 (Natural deduction NJ). Propositional formulas are the types of Definition 10.1, read with $\rightarrow, \wedge, \vee, \perp$ for $\rightarrow, \times, +, \mathbf{0}$. The sequent calculus NJ has the rules: assumption; $\rightarrow$ I (discharging an assumption) and $\rightarrow$ E (modus ponens); $\wedge$ I and the two $\wedge$ E; the two $\vee$ I and $\vee$ E (case analysis, discharging an assumption in each branch); and $\perp$ E (from $\perp$ infer any formula).

Theorem 10.3 (Curry–Howard correspondence for $\lambda^{\rightarrow \times + \mathbf{0}}$ and NJ). *The statement concerns exactly the calculus of Definition 10.1 and the system NJ of Definition 10.2, and nothing larger. (a) If $\Gamma \vdash t : A$ is derivable, then $\underline{\Gamma} \vdash A$ is derivable in NJ, where $\underline{\Gamma}$ is the set of types in Γ . (b) If $\Delta \vdash A$ is derivable in NJ, then there are a context Γ listing the formulas of Δ with distinct variables and a term t with $\Gamma \vdash t : A$.*

Proof. The typing rules of Definition 10.1 are in bijection with the rules of NJ: variable with assumption; abstraction with $\rightarrow$ I; application with $\rightarrow$ E; pairing and projections with $\wedge$ I, $\wedge$ E; inl, inr with $\vee$ I; **case** with $\vee$ E; **abort** with $\perp$ E. Forgetting the terms in a typing derivation yields an NJ derivation, which proves (a). Conversely, given an NJ derivation, assign a fresh variable to each assumption and build the term by following the derivation rule by rule; this proves (b), by induction on the derivation. $\square$

Example 10.4 (A detour and its removal). The term $(\lambda x:A. \langle x, x \rangle) a$ of type $A \times A$ corresponds to a derivation that introduces $A \rightarrow A \times A$ by $\rightarrow$ I and immediately eliminates it by $\rightarrow$ E. The reduction $(\lambda x:A. \langle x, x \rangle) a \rightarrow \langle a, a \rangle$ replaces that detour by the direct derivation that uses the derivation of A twice. In general, a β -step is the removal of an introduction immediately followed by the matching elimination.

10.3 Normalization and consistency

Cited result 10.5 (Normalization). Reduction in $\lambda^{\rightarrow \times + \mathbf{0}}$ is strongly normalizing (no infinite reduction sequence) and preserves types (subject reduction) [28, 22].

Lemma 10.6 (Closed normal terms are introductions). *A closed term in normal form is an abstraction, a pair, or an inl or inr term. In particular no closed normal term has type $\mathbf{0}$.*

Proof. Induction on the term t . It cannot be a variable, since it is closed. If t is an elimination form, namely $f a$, $\pi_i p$, **case** s of $\dots$, or **abort** _{C} e , then its principal argument (f , p , s , or e) is a closed subterm of a normal term, hence closed and normal, hence by the induction hypothesis an abstraction, a pair, or an **inl**/**inr** term. For $f a$, $\pi_i p$, and **case** this makes t a redex, contradicting normality. For **abort** _{C} e with e of type $\mathbf{0}$, an introduction form for $\mathbf{0}$ does not exist, since the types of abstractions, pairs, and **inl**/**inr** terms are $\rightarrow$, $\times$, and $+$. So t must be an introduction. No introduction has type $\mathbf{0}$. $\square$

Theorem 10.7 (Consistency of the object calculus). *There is no closed term of type $\mathbf{0}$. Hence NJ does not derive $\perp$ from no assumptions.*

Proof. Suppose t is closed of type $\mathbf{0}$. By Cited result 10.5, t reduces to a normal form t' , which is closed and, by subject reduction, of type $\mathbf{0}$, contradicting Lemma 10.6. For the second claim use Theorem 10.3(b). $\square$

Remark 10.8 (Object calculus and metatheory). Theorem 10.7 is a statement *about* the calculus $\lambda^{\rightarrow \times + 0}$, proved *in the metatheory* with the help of the normalization theorem of Cited result 10.5. It does not say that the calculus proves its own consistency: the calculus has no way to express a statement about its own terms, and the normalization argument is carried out outside it. There is no tension with Gödel's second theorem (Cited result 9.1), which concerns consistent theories that interpret elementary arithmetic; $\lambda^{\rightarrow \times + 0}$ has no natural numbers and does not interpret arithmetic. For the dependent kernel of Section 11, which does contain **Nat**, the corresponding statement is relative: its consistency is established from a stronger metatheory (Cited result 11.8, stated in that section), and, by the same theorem, cannot be established inside it.

Corollary 10.9 (No excluded middle). *For a base type α there is no closed term of type $\alpha + (\alpha \rightarrow \mathbf{0})$.*

Proof. A closed term of this type would give, by Theorem 10.3(a), an NJ derivation of $\alpha \vee \neg \alpha$, which by Cited result 5.2 would be valid in every open-world semantics. Example 5.5 gives a valuation (take $X = \mathbb{R}$ and $\llbracket \alpha \rrbracket = \mathbb{R} \setminus \{0\}$) at which it is not valid. $\square$

Corollary 10.9 joins two earlier parts of this text: a topological counterexample to excluded middle (Section 5) forbids the existence of a program of the corresponding type. Classical principles are available only by adding a constant, as in Section 12.

10.4 What the dependent kernel adds

The rule for $A \rightarrow B$ is the case of $\Pi x:A. B$ in which B does not mention x ; $A \times B$ is the corresponding case of a dependent pair; and the empty type remains the type with no constructors. The new ingredients of Section 11 are (i) types depending on terms, so that quantifiers $\forall x:A. P(x)$ are Π -types, (ii) a universe **Prop** of propositions with proof irrelevance, and (iii) inductive types in general. The correspondence of Theorem 10.3 then extends to a correspondence between first-order-style intuitionistic logic and the Π fragment, which is cited here and not proved [24, 27].

11 Refuse inside Bind: the kernel of a dependent type theory

11.1 What changes relative to the term layer

Section 3 ended with a limitation: every term applies to every term. A dependent type theory removes the limitation by giving every binder a *domain*, and by allowing the range of the body to depend on the pop being bound. The restriction of a domain, which in Part III was a predicate applied to a prior set from outside, becomes a construction inside the calculus. The presentation below follows the kernel of Lean 4 as described by Carneiro [5] and de Moura and Ullrich [15]; it is a summary of known rules, not a new system.

11.2 Judgments and the rules

Contexts are lists $\Gamma ::= \cdot \mid \Gamma, x:A$. There are three judgments: $\Gamma \vdash t : A$, $\Gamma \vdash A \equiv B$ (definitional equality), and well-formedness of contexts, which is left implicit. Universe levels are generated by 0, successor, max, imax, and level variables, where

$$\text{imax}(\ell, 0) = 0, \quad \text{imax}(\ell, m + 1) = \max(\ell, m + 1).$$

$\text{Sort } \ell$ is the universe of level ℓ ; $\text{Prop} := \text{Sort } 0$ and $\text{Type } \ell := \text{Sort}(\ell + 1)$.

$$\begin{array}{c} \frac{}{\Gamma \vdash \text{Sort } \ell : \text{Sort}(\ell + 1)} \text{ (Sort)} \quad \frac{x:A \in \Gamma}{\Gamma \vdash x : A} \text{ (Pop)} \\[10pt] \frac{\Gamma \vdash A : \text{Sort } i \quad \Gamma, x:A \vdash B : \text{Sort } j}{\Gamma \vdash \Pi x:A. B : \text{Sort } \text{imax}(i, j)} \text{ (II)} \quad \frac{\Gamma, x:A \vdash t : B}{\Gamma \vdash \lambda x:A. t : \Pi x:A. B} \text{ (BIND)} \\[10pt] \frac{\Gamma \vdash f : \Pi x:A. B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B[x := a]} \text{ (COLLAPSE)} \quad \frac{\Gamma \vdash t : A \quad \Gamma \vdash A \equiv B}{\Gamma \vdash t : B} \text{ (Conv)} \end{array}$$

Definitional equality is the least congruence containing: β , $(\lambda x:A. t) a \equiv t[x := a]$; η , $\lambda x:A. f x \equiv f$ for x not free in f ; δ and ζ (unfolding of definitions and of local definitions); ι (reduction of a recursor applied to a constructor); and *proof irrelevance*:

$$\frac{\Gamma \vdash P : \text{Prop} \quad \Gamma \vdash p : P \quad \Gamma \vdash q : P}{\Gamma \vdash p \equiv q : P} \text{ (PI)}$$

Analogy 11.1 (Operators in the kernel). The rule (Pop) introduces a pop already present in the context. The rule (Bind) abstracts one. The rule (Collapse) is application, whose computational content is the β rule of Section 3, now applied to terms whose domains are checked. (PI) is a Collapse that identifies all proofs of a proposition. Refuse has no rule of its own; it is built from the others, as the next subsection shows.

11.3 Refuse as subtype formation

A *predicate* on A is a term $p : A \rightarrow \text{Prop}$. Mathlib-style sets are predicates: $\text{Set } A := A \rightarrow \text{Prop}$, with $a \in S$ meaning $S a$.

Definition 11.2 (Internal Refuse). For $A : \text{Sort } u$ and $P : A \rightarrow \text{Prop}$, the *refused domain* is the subtype

$$\text{Refuse}_P(A) := \{x : A \mid \neg P x\}, \quad \neg Q := Q \rightarrow \text{False}.$$

The elements of $\{x : A \mid Q x\}$ are pairs $\langle a, h \rangle$ with $a : A$ and $h : Q a$.

Proposition 11.3 (Refuse laws survive internally). *For A a type and $P, Q : A \rightarrow \mathbf{Prop}$: (a) if P is the constant predicate $\lambda x. \mathbf{True}$ then $\mathbf{Refuse}_P(A)$ has no inhabitants (is empty); (b) if $P = \lambda x. \mathbf{False}$ then the projection $\mathbf{Refuse}_P(A) \rightarrow A$ has a right inverse, so every element of A yields an element of $\mathbf{Refuse}_P(A)$.*

Proof. (a) An inhabitant would be $\langle a, h \rangle$ with $h : \neg \mathbf{True}$, that is $h : \mathbf{True} \rightarrow \mathbf{False}$. The function $\lambda \langle a, h \rangle. h$ trivial has type $\mathbf{Refuse}_P(A) \rightarrow \mathbf{False}$, so an inhabitant of the subtype would yield a term of $\mathbf{False}$; in a consistent theory there is none. (b) Given $a : A$, the pair $\langle a, \lambda h. h \rangle$ has second component of type $\neg \mathbf{False} = \mathbf{False} \rightarrow \mathbf{False}$, which is inhabited by the identity. $\square$

Proposition 11.3(a) is the type-theoretic reading of “ $\mathbf{Refuse}_{\mathbf{U}}(\Omega) = \emptyset$ ” (Proposition 2.2(iii)); it is a statement that the subtype is uninhabited, proved by exhibiting a function into $\mathbf{False}$. (b) is the reading of $\mathbf{Refuse}_{\emptyset}(\Omega) = \Omega$.

Remark 11.4. The passage from external to internal Refuse is the point at which the Domain-first rule (Design rule 8.4) becomes automatic: the subtype constructor takes the type A as an argument, so there is no way to write a refusal without a domain. This is a syntactic fact about the formation rule, not a theorem about consistency.

11.4 Impredicativity of Prop

By the (II) rule with $j = 0$, $\text{imax}(i, 0) = 0$, so a product $\Pi x:A. B$ with $B : \mathbf{Prop}$ lies in $\mathbf{Prop}$ *whatever the universe i of A* . Consequently one may quantify over all predicates, $\forall S : \mathbf{Set} A, \dots$, and remain in $\mathbf{Prop}$.

Analogy 11.5. This is the type-theoretic counterpart of the Power set axiom’s quantification over all refusals (Proposition 8.8). In both cases a statement is allowed to range over the full supply of refusals, including those that are not definable by any finite expression. In ZFC the supply is a set $\mathcal{P}(\Omega)$; in type theory the supply is the type $A \rightarrow \mathbf{Prop}$ and quantification over it is permitted inside $\mathbf{Prop}$ itself.

11.5 Inductive types and the empty type

An inductive type is introduced by its constructors and eliminated by a recursor, with ι -reduction as the computation rule. Examples relevant here are $\mathbb{N}$ (constructors $0, \text{succ}$), $\mathbf{Eq}$, and the types with no constructors.

Definition 11.6 (Empty types). $\mathbf{Empty} : \mathbf{Type}$ and $\mathbf{False} : \mathbf{Prop}$ are the inductive types with no constructors. Their recursors are $\mathbf{Empty.rec} : \Pi(e : \mathbf{Empty}). C e$ and $\mathbf{False.rec} : \Pi(h : \mathbf{False}). C h$, for any motive C .

Elimination from $\mathbf{False}$ into an arbitrary sort is allowed. This is an instance of the general restriction that an inductive proposition may eliminate only into $\mathbf{Prop}$ unless it is a *subsingleton* in a sense given in [5] (roughly: at most one constructor, whose arguments are either propositions or determined by the output indices); $\mathbf{False}$, with no constructors, satisfies it vacuously. The restriction is what keeps proof irrelevance from being refuted by case analysis on proofs.

Proposition 11.7 (Negation and inhabitation). *For any type A , $(A \rightarrow \mathbf{False}) \leftrightarrow \neg \mathbf{Nonempty} A$, where $\mathbf{Nonempty} A : \mathbf{Prop}$ is the inductive proposition with the single constructor $\text{intro} : A \rightarrow \mathbf{Nonempty} A$.*

Proof. $(\Rightarrow)$ Given $f : A \rightarrow \text{False}$ and $\text{intro } a : \text{Nonempty } A$, eliminate $\text{intro } a$ into False (a proposition, so elimination from Nonempty is permitted) to get $f a : \text{False}$. $(\Leftarrow)$ Given $h : \neg \text{Nonempty } A$ and $a : A$, $h(\text{intro } a) : \text{False}$. $\square$

The proposition separates *emptiness of a type* (no term of type A) from *the proposition* $\neg \text{Nonempty } A$. Both are encoded here by the same function into False . In set theory, by contrast, $\emptyset$ is a particular set (Proposition 8.2); in type theory, “empty” is a property of a type or of a predicate. The empty subworld of A is the predicate $\lambda x. \text{False} : \text{Set } A$, which is the internal version of $\text{Refuse}_{\mathbb{U}}(\Omega)$.

Cited result 11.8 (Normalization and consistency). For the calculus of constructions and its extension with inductive types, closed well-typed terms normalize and False has no closed inhabitant [7, 8, 19]. For the Lean kernel, consistency is established relative to ZFC with inaccessible cardinals [5]; *decidability of definitional equality and subject reduction are known to be delicate for the full system*, because of the interaction of proof irrelevance, quotients, and accessibility-based recursion, and are discussed in the same source.

11.6 A worked derivation

The kernel rules are exercised on the smallest nontrivial case: the natural numbers and addition. The derivations are carried out by hand at the level of the rules of Section 11.

Definition 11.9 (Naturals, equality, addition). $\text{Nat} : \text{Type}$ is the inductive type with constructors $\text{zero} : \text{Nat}$ and $\text{succ} : \text{Nat} \rightarrow \text{Nat}$. Its recursor is

$$\text{Nat.rec} : \Pi(C : \text{Nat} \rightarrow \text{Sort } u). C \text{ zero} \rightarrow (\Pi n : \text{Nat}. C n \rightarrow C(\text{succ } n)) \rightarrow \Pi n : \text{Nat}. C n,$$

with ι -reductions

$$\text{Nat.rec } C \text{ z s zero} \equiv z, \quad \text{Nat.rec } C \text{ z s (succ } n) \equiv s n (\text{Nat.rec } C \text{ z s } n).$$

$\text{Eq} : \Pi\{A : \text{Sort } u\}, A \rightarrow A \rightarrow \text{Prop}$ is the inductive proposition with one constructor $\text{Eq.refl } a : \text{Eq } a a$; its recursor is the transport principle Eq.rec , with $\text{Eq.rec } m (\text{Eq.refl } a) \equiv m$. Write $\text{congrArg } f h := \text{Eq.rec } (\text{Eq.refl } (f a)) h : \text{Eq } (f a) (f b)$ for $f : A \rightarrow B$ and $h : \text{Eq } a b$. Addition is defined by recursion on its second argument:

$$\text{add} := \lambda m n : \text{Nat}. \text{Nat.rec } (\lambda _ . \text{Nat}) m (\lambda _ r. \text{succ } r) n.$$

Write $1 := \text{succ zero}$, $2 := \text{succ } 1$, $4 := \text{succ } (\text{succ } 2)$.

Proposition 11.10 (Computation proves $2 + 2 = 4$). $\vdash \text{Eq.refl } 4 : \text{Eq } (\text{add } 2 2) 4$.

Proof. By the rule for Eq.refl , $\text{Eq.refl } 4 : \text{Eq } 4 4$. By the conversion rule it suffices to show $\text{add } 2 2 \equiv 4$:

$$\begin{aligned} \text{add } 2 2 &\equiv \text{Nat.rec } (\lambda _ . \text{Nat}) 2 s (\text{succ } 1) && (\delta, \beta) \\ &\equiv s 1 (\text{Nat.rec } (\lambda _ . \text{Nat}) 2 s 1) && (\iota) \\ &\equiv \text{succ } (\text{Nat.rec } (\lambda _ . \text{Nat}) 2 s (\text{succ zero})) && (\beta) \\ &\equiv \text{succ } (s \text{ zero } (\text{Nat.rec } (\lambda _ . \text{Nat}) 2 s \text{ zero})) && (\iota) \\ &\equiv \text{succ } (\text{succ } (\text{Nat.rec } (\lambda _ . \text{Nat}) 2 s \text{ zero})) && (\beta) \\ &\equiv \text{succ } (\text{succ } 2) = 4 && (\iota), \end{aligned}$$

where $s := \lambda _ r. \text{succ } r$. $\square$

Here the kernel decides the equation by computation, with no proof search. The same mechanism proves more than numerical instances.

Proposition 11.11 ($n + 0 = n$ by computation).

$$\vdash \lambda n:\text{Nat}. \text{Eq.refl } n : \Pi n:\text{Nat}. \text{Eq} (\text{add } n \text{ zero}) n.$$

Proof. For a variable n , $\text{add } n \text{ zero} \equiv \text{Nat.rec} (\lambda _.\text{Nat}) n s \text{ zero} \equiv n$ by ι , since the recursion is on the *second* argument and that argument is the constructor `zero`. So $\text{Eq.refl } n$ has the required type by conversion. $\square$

The mirror equation does not follow from computation.

Proposition 11.12 ($0 + n = n$ needs induction).

$$\vdash \lambda n:\text{Nat}. \text{Nat.rec} (\lambda k. \text{Eq} (\text{add zero } k) k) (\text{Eq.refl zero}) \\ (\lambda k h. \text{congrArg succ } h) n : \Pi n:\text{Nat}. \text{Eq} (\text{add zero } n) n.$$

Moreover, $\text{add zero } n \not\equiv n$ for a variable n , so $\lambda n.\text{Eq.refl } n$ does not have this type.

Proof. Take the motive $C k := \text{Eq} (\text{add zero } k) k$. *Base:* $C \text{ zero} = \text{Eq} (\text{add zero zero}) \text{ zero} \equiv \text{Eq zero zero}$ by ι , which is inhabited by Eq.refl zero . *Step:* given $h : C k$, that is $h : \text{Eq} (\text{add zero } k) k$, the goal is $C (\text{succ } k) = \text{Eq} (\text{add zero } (\text{succ } k)) (\text{succ } k)$, and by ι the left side is $\equiv \text{succ} (\text{add zero } k)$. Then $\text{congrArg succ } h : \text{Eq} (\text{succ} (\text{add zero } k)) (\text{succ } k)$ has the required type. Applying the recursor to n gives the stated term.

For the final claim: $\text{add zero } n \equiv \text{Nat.rec} (\lambda _.\text{Nat}) \text{zero } s n$ with n a variable. No ι -rule applies because n is not a constructor application, so this term is in normal form with a head that is a recursor stuck on a variable, and it is not the variable n . By confluence of reduction for the system (Cited result 11.8) two terms with distinct normal forms are not definitionally equal. $\square$

Proposition 11.13 (Footprint). *The three terms of Propositions 11.10, 11.11, and 11.12 use only the rules of Section 11 and the inductive types Nat and Eq, and declare no axioms. (In the terminology introduced in Section 12, their footprints, Definition 12.2, are empty.)*

Proof. By inspection: no declared axiom occurs in the terms or in the definitions they unfold. $\square$

The pair of results shows, in the smallest setting, the distinction on which the later parts rely. Whether an equation is available by *computation* depends on how the function is defined, not only on the function: $n + 0 = n$ holds by computation and $0 + n = n$ by induction, although they are equally true of the same function. The kernel is a decision procedure for definitional equality, and everything beyond it must be supplied as a proof term.

11.7 Quotient types as Collapse

For a type A and a relation $r : A \rightarrow A \rightarrow \text{Prop}$ the kernel provides $\text{Quot } r$, the constructor $\text{Quot.mk} : A \rightarrow \text{Quot } r$, the eliminator Quot.lift , and the principle Quot.sound :

$$\text{Quot.lift } f h (\text{Quot.mk } a) \equiv f a \ (\iota), \quad \text{Quot.sound} : r a b \rightarrow \text{Quot.mk } a = \text{Quot.mk } b,$$

where $f : A \rightarrow B$ and h is a proof that f respects r .

Cited result 11.14 (Funext from Quot.sound). Function extensionality is derivable from Quot.sound (and propositional extensionality is not, and is taken as a separate axiom). See [5].

Proposition 11.15 (Quot.lift is the universal property of Collapse). *Let $f : A \rightarrow B$ respect r . Then $g := \text{Quot.lift } f \ h$ satisfies $g \circ \text{Quot.mk} \equiv f$ definitionally, and any $g' : \text{Quot } r \rightarrow B$ with $g' \circ \text{Quot.mk} = f$ (propositionally, pointwise) satisfies $g' = g$ given function extensionality.*

Proof. The first claim is the ι -rule. For the second, Quot.ind reduces equality of two functions on Quot r to equality of their composites with Quot.mk, and function extensionality (Cited result 11.14) turns pointwise equality into equality of functions. $\square$

Proposition 11.15 is Proposition 2.3 read inside the kernel. The domain-layer quotient and the kernel quotient have the same universal property; the kernel version has the extra data of a proof h because “constant on classes” is itself a proposition that must be supplied.

11.8 Sets inside types

Definition 11.16 (Pre-sets). Following Aczel [2], define the inductive type PSet whose constructor takes an index type I and a family $f : I \rightarrow \text{PSet}$. Equip it with a bisimulation-style equivalence $\simeq$ and set $\text{ZFSet} := \text{Quot}(\simeq)$. Membership is defined through the family.

This construction is used in the Mathlib library for the type of ZF sets [16]. It places a model of ZF(C)-style set theory *inside* a type theory. The reverse direction, a model of type theory inside set theory, is the subject of the next section.

12 Axioms added to the kernel

12.1 Three standard additions

The Lean kernel as summarized in Section 11 is not classical. Three statements are in common use on top of it:

- (i) Quot.sound, which is a primitive of the kernel, as above;
- (ii) *propositional extensionality*, $\text{propext} : \Pi(p\ q : \text{Prop}). (p \leftrightarrow q) \rightarrow p = q$;
- (iii) *global choice*, $\text{Classical.choice} : \Pi(\alpha : \text{Sort } u). \text{Nonempty } \alpha \rightarrow \alpha$.

Each is a revision of what the system admits (Section 14).

12.2 Choice carries classical logic

Cited result 12.1 (Diaconescu, internal form). From Classical.choice and propext one derives $\Pi(p : \text{Prop}). p \vee \neg p$.

Sketch. Let $Ux := (x = \text{true}) \vee p$ and $Vx := (x = \text{false}) \vee p$ on Bool. The subtypes $\{x \mid Ux\}$ and $\{x \mid Vx\}$ are nonempty, so choice supplies a and b with $Ua.1$ and $Vb.1$. Case-splitting on the disjunctions gives either p , or $a.1 = \text{true}$ and $b.1 = \text{false}$. If p held, U and V would be propositionally equal predicates (each is equivalent to True, so propext

and function extensionality identify them), and choice applied to equal arguments would give equal results, so $a.1 = b.1$. Hence $p \rightarrow a.1 = b.1$, and in the second case $\neg p$. The use of `propext` in the middle of the argument is the type-theoretic counterpart of the use of Extensionality in Theorem 8.11. $\square$

12.3 The axiom footprint

Definition 12.2 (Footprint). For a closed declaration d , its *footprint* $\text{Ax}(d)$ is the set of declared axioms that occur in d or, recursively, in any declaration that d uses.

Proposition 12.3 (Footprints are monotone). *If d uses d' then $\text{Ax}(d') \subseteq \text{Ax}(d)$. If $\text{Ax}(d) = \emptyset$, then d is a theorem of the kernel alone.*

Proof. Both statements are immediate from the recursive definition: the footprint of d is the union of the axioms occurring in d with the footprints of the declarations d uses, and a term whose footprint is empty is, after unfolding, a term built from the kernel rules alone. $\square$

Footprints make precise a distinction that is often blurred. Whether a statement “needs choice” is a property of a *particular proof term*, not of the statement. A function that selects the first element of a nonempty list has an empty footprint, because the list supplies the selection by data; a function that selects an element from a type known only to be `Nonempty` has `Classical.choice` in its footprint, because nothing supplies the selection. Both are “choose an element of a nonempty collection,” and only the second is the Axiom of Choice in the sense of Proposition 8.9.

Design rule 12.4 (Report footprints). A formal development should state the footprint of each headline theorem, in the manner of a list of axioms, so that a reader can see which of the revisions of Section 14 the result depends on.

13 Translating between sets and types

13.1 What is known

Cited result 13.1 (Relating the two worlds). (a) Constructive set theory CZF is interpretable in Martin-Löf type theory with a universe via the `PSet` construction of Definition 11.16 [2, 3].

(b) The calculus of inductive constructions with n universes and classical axioms is equiconsistent with ZFC augmented by n inaccessible cardinals, for finite n [20].

(c) The type theory of Lean is consistent relative to ZFC with countably many inaccessible cardinals, by an interpretation in which `Sort  $\ell$` is modelled by V_{κ_ℓ} for inaccessibles $\kappa_0 < \kappa_1 < \dots$ [5].

The interpretation in (b) and (c) matches the earlier remark that each level of the universe hierarchy corresponds to an additional inaccessible (Cited result 8.12): “another universe” is an extra admissibility rule, consistently addable, never derivable from the lower levels. A foundation with finitely many universes is weaker than one with infinitely many, and the difference is exactly the strength of the inaccessibles added.

13.2 The second dictionary

	ZFC	Dependent type theory
Where is Refuse?	Outside the objects: a formula applied to a set (Separation)	Inside the calculus: a predicate $A \rightarrow \mathbf{Prop}$ and a subtype
What is membership?	A primitive relation $\in$ among sets	A typing judgment $t : A$ at the meta level; $a \in S := S a$ for predicates
What is equality?	Extensionality axiom (a statement about sets)	Identity type, with $\beta\eta$ -conversion and proof irrelevance built into the kernel
Collapse	Extensionality, equivalence classes, Replacement	β , $\mathbf{Quot}$, proof irrelevance
Impredicative quantification	Power set	Π into $\mathbf{Prop}$
Choice	An axiom	$\mathbf{Classical.choice}$, an axiom outside the kernel
Universe growth	Inaccessible cardinals	Universe levels

Analogy 13.2. Read through this table, the three foundations of the title are three answers to a single question: *where is a domain allowed to be restricted, and who checks the restriction?* ZFC answers “from outside, by a formula, over a set already given.” Constructive logic answers “inside, by an admissible region that must be stable under refinement.” A dependent type theory answers “inside, by a type former whose arguments carry the domain, checked by the kernel.” The table does not prove any equivalence between them; the equivalences are those of Cited result 13.1.

13.3 Libraries and kernels

A library built on the kernel can contain several coexisting representations of the same informal notion. A “set of elements of A ” may be a predicate $A \rightarrow \mathbf{Prop}$ (internal Refuse) or an element of $\mathbf{ZFSet}$ (an Aczel-style quotient). These are different objects, related by a translation that is itself a theorem of the library, not an identification made by the kernel. A statement about one does not automatically transfer to the other. This matters for any claim about what “Lean’s sets” are: the answer depends on which of the two representations is meant.

Part V

Reflection

14 Revising the admissibility rule

14.1 The operator A

The four operators act within a fixed rule of admissibility. Foundations change when the rule itself changes.

Definition 14.1 (Revision). A *rule-set* is a theory T in a fixed language (axioms and inference rules). A *revision* A is a map from rule-sets to rule-sets. The revisions considered here add an axiom: $A_\sigma(T) = T + \sigma$.

Examples already encountered are $\sigma = \text{AC}$ over ZF; $\sigma = \text{Classical.choice}$ or $\sigma = \text{propext}$ over the kernel; and $\sigma = \text{“an inaccessible cardinal exists”}$ over ZFC, with its type-theoretic counterpart, one more universe level.

Proposition 14.2 (Revision and independence). *If T is consistent and $T \not\vdash \sigma$ and $T \not\vdash \neg\sigma$, then $A_\sigma(T)$ and $A_{\neg\sigma}(T)$ are both consistent and neither is contained in the other.*

Proof. If $T + \sigma$ were inconsistent then $T \vdash \neg\sigma$, contrary to hypothesis; symmetrically for $T + \neg\sigma$. If $T + \sigma \supseteq T + \neg\sigma$ in provable consequences then $T + \sigma \vdash \neg\sigma$, so $T + \sigma$ would be inconsistent; and conversely. $\square$

By Cited result 8.10, AC over ZF satisfies the hypotheses of Proposition 14.2 whenever ZF is consistent.

Combined with Cited result 8.12, this says that no sufficiently strong foundation can certify from within the soundness of a revision that raises its own consistency strength. Statements of the form “this system is justified” are therefore always made from a stronger vantage point, or are not made at all. The honest form of a foundational claim is relative: *this theorem is derivable in T , whose footprint is F , and T is consistent relative to T' .*

15 Where a statement can fail

A formal statement passes through several stages before it counts as established, and it can fail at each. The classification below is generic and applies to any proof assistant with a kernel.

Definition 15.1 (Failure layers). (1) *Parsing*: the text is not a well-formed expression of the surface language.

(2) *Elaboration*: the expression is parsed but cannot be completed to a well-typed term; implicit arguments, coercions, universe levels, or the expected sort of a position cannot be resolved.

(3) *Tactic or proof search*: elaboration succeeds but a proof step does not close its goal.

(4) *Kernel*: a fully elaborated term is rejected by the independent checker.

(5) *Derived contradiction*: the term is accepted, and what it proves is inconsistent with the intended reading, for example because an assumption is inconsistent.

(6) *Scope*: everything is accepted and consistent, but the theorem does not state what its informal description says it states.

Example 15.2 (Two elaboration failures). (i) A position that requires a proposition is given a value: $\forall n : \mathbb{N}, n \leq 2 \rightarrow 0$. The final 0 has type $\mathbb{N}$, but the arrow expects a type or a proposition there. This fails at layer (2). (ii) A proposition is passed to a function on booleans: with $\text{not} : \text{Bool} \rightarrow \text{Bool}$ and $\neg : \text{Prop} \rightarrow \text{Prop}$ distinct, the expression $\text{not}(n \leq 2)$ expects a Bool and receives a Prop . This also fails at layer (2). The usual repair is to use $\neg$, or to insert a decision procedure that converts the proposition to a boolean.

Analogy 15.3 (Proof checking as a fifth operator). Layers (2) through (4) together play the role of a *Verify* step that follows *Bind* and precedes *Collapse* in the lifecycle used elsewhere in the Spheredrop program: a term is built, its admissibility is checked, and only then is it evaluated or identified. This is a resemblance of order only.

The layers are ordered by how much of the argument has been tested: a layer-(1) or (2) failure says nothing about the mathematics, because the statement was never in a form the system could examine; a layer-(5) failure says the theorem is vacuous or false as stated; a layer-(6) failure says the formalization is correct and its meaning is not. Reports of “verified” results should name the layer at which the check was passed.

Proposition 15.4 (A verified theorem is relative to its axioms). *If d is accepted by the kernel with footprint $F = \text{Ax}(d)$, then d is a theorem of the kernel extended by F . In particular, a statement obtained as a theorem about hypotheses supplied as declared axioms is a theorem of the form “if the axioms then the conclusion,” and its footprint contains those axioms.*

Proof. By Proposition 12.3, every axiom used by d lies in F , so d is derivable in the kernel with F added. For the second sentence, an axiom α declared by the author and used in the proof of d occurs in $\text{Ax}(d)$ by definition. $\square$

Proposition 15.4 is elementary and is the formal content of the slogan that a derivation cannot be more informative than its premises. It is worth stating because it separates two readings of a verified result: *the conclusion holds* and *the conclusion follows from the declared axioms*. Only the second is established.

16 Conventions as a common language

16.1 The role of an arbitrary base

The axioms of ZFC are not forced. Independence results (Cited result 8.10) show that Choice can be added or its negation added, with both extensions consistent if ZF is. The same is true of the Continuum Hypothesis over ZFC [6]. In that precise sense the choice of base is a convention, comparable to the choice of a unit of length: a different unit would do equally well as far as nature is concerned, and no measurement is more correct for being in metres.

What a convention provides is not truth but *comparability*. Two parties who have adopted the same base can exchange statements and derivations, and each can check what the other produced. The value of ZFC, and of a fixed kernel with a fixed set of added axioms, is that it supplies one rule of admissibility, stated in advance, that everyone can apply. The rest of this section makes the point exact.

16.2 Reproducible results

Definition 16.1 (Reproducible result). Let T be a theory with a decidable set of axioms. A *reproducible result relative to T* is a triple (T, φ, d) where d is a finite derivation of the sentence φ from the axioms of T .

Cited result 16.2 (Checkability). For a first-order theory with a decidable set of axioms, the relation “ d is a derivation of φ from T ” is decidable. See Enderton [10, Ch. 2 and 3].

Cited result 16.2 is the logical content of the word *reproducible* in this text: given the base, the question whether a purported derivation is correct has a mechanical answer that does not depend on who asks. The kernel of a proof assistant is an implementation of this decision procedure for a particular base. (For the full Lean kernel the situation is subtler, as noted in Cited result 11.8, and this text claims only what the cited sources claim.)

Proposition 16.3 (Results compose over a common base). *(a) If (T, φ, d) and $(T, \varphi \rightarrow \psi, e)$ are reproducible results then so is $(T, \psi, d * e)$, where $d * e$ concatenates the derivations and applies modus ponens. (b) If $T \subseteq T'$ and (T, φ, d) is reproducible, then (T', φ, d) is reproducible.*

Proof. (a) The concatenation is a finite sequence each of whose lines is an axiom of T or follows from earlier lines, ending in ψ . (b) Every axiom of T is an axiom of T' , so d is a derivation from T' . $\square$

Proposition 16.3(a) is what a common base buys: derivations from the same T can be chained without translation. Part (b) says that results are portable *upward*: a result proved over a weaker base remains valid over any stronger one. The converse fails, which is why a footprint matters (Definition 12.2).

Example 16.4 (Results that cannot be combined). Let $T_1 = \text{ZF} + \text{AC}$ and $T_2 = \text{ZF} + \neg\text{AC}$. If ZF is consistent, each of T_1 and T_2 is consistent (Cited result 8.10), so each supports many reproducible results. But $T_1 \cup T_2$ proves both AC and $\neg\text{AC}$ and so proves every sentence. A reproducible result relative to T_1 and one relative to T_2 cannot be chained in either theory in the manner of Proposition 16.3(a), and the pair says nothing jointly. A community in which half the authors assumed T_1 and half T_2 , without saying so, would produce correct and mutually unusable results.

Example 16.4 is the formal reason that an *arbitrary* base is better than a different base for each author. The choice among conventions is not forced, but the existence of one agreed convention is what lets results be collected, compared, and reused.

Analogy 16.5 (Units, formats, and protocols). A unit of length, a character encoding, and a network protocol share one feature: any particular choice could have been otherwise, and what matters is that a single choice is widely adopted and written down. Foundations of mathematics are a convention of this kind with an unusually long track record. The analogy proves nothing about mathematics; it says what role the convention plays.

17 Arbitrary bases, determinate consequences

17.1 Arbitrary at the base, forced above it

That the base is a convention does not make its consequences conventional. Once a base T is fixed, whether $T \vdash \varphi$ is a matter of fact, settled by a derivation or by an independence proof. Two layers must be kept apart:

- the *choice of T* is not forced and is not true or false;
- *what follows from T* is determinate, and statements about it are true or false.

A claim of the form “the Axiom of Choice is arbitrary” is correct about the first layer and says nothing about the second. A claim of the form “therefore anything derived from it is arbitrary” confuses the two.

17.2 How much does the convention matter?

The choice matters little for some statements and a great deal for others, and both facts are theorems.

Cited result 17.1 (Conservativity for arithmetic). ZF and ZFC prove the same Π_2^1 sentences; in particular they prove the same arithmetical sentences. This follows from Gödel's constructible universe together with Shoenfield absoluteness. See Jech [13, Ch. 13 and Ch. 25].

So adopting Choice never changes which arithmetical statements are provable. It changes what can be proved about large infinite objects.

Cited result 17.2 (Choice and measurability). In ZFC there is a subset of $\mathbb{R}$ that is not Lebesgue measurable (Vitali). Relative to the consistency of an inaccessible cardinal, ZF with dependent choice is consistent with every set of reals being Lebesgue measurable (Solovay [17]).

Cited result 17.2 shows that the choice of convention can have visible consequences in analysis. It also shows that the consequence is itself determinate: given the base, the answer follows.

17.3 What a convention can be assessed on

Design rule 17.3 (Assessing a convention). A foundational base is assessed on four things, none of which is truth:

- (i) *Consistency relative to other bases* (Section 13);
- (ii) *Strength*: whether the mathematics to be done can be carried out in it;
- (iii) *Conservativity*: which statements it leaves unchanged relative to a weaker base, as in Cited result 17.1;
- (iv) *Adoption*: how many parties use it, since that is what gives comparability.

Remark 17.4. This does not make the choice of base unconstrained. A base that is inconsistent has no use as a common language, since it proves every sentence. A base too weak to express the statements parties wish to exchange is not a common language for them. Adoption alone decides between bases that satisfy the other three equally well.

18 One object, many representations: the radix example

18.1 Representation systems and canonical forms

The earlier sections distinguish a convention from what follows from it. A numeral system is the simplest place to see the distinction proved rather than asserted, because the object represented (a natural number) is fixed in advance and the representation is a free choice.

Definition 18.1 (Representation system). A *representation system* is a triple $(R, \llbracket \cdot \rrbracket, \rightarrow)$ of a set R of inscriptions, a *denotation* map $\llbracket \cdot \rrbracket : R \rightarrow X$ into a set X of represented objects, and a rewriting relation $\rightarrow$ on R . It is *sound* if $r \rightarrow r'$ implies $\llbracket r \rrbracket = \llbracket r' \rrbracket$, *terminating* if there is no infinite chain $r_0 \rightarrow r_1 \rightarrow \dots$, and *confluent* if $r \twoheadrightarrow s$ and $r \twoheadrightarrow t$ imply a common u with $s \twoheadrightarrow u$ and $t \twoheadrightarrow u$. A *normal form* is an inscription with no outgoing rewrite.

Theorem 18.2 (Canonical forms). *Let $(R, \llbracket \cdot \rrbracket, \rightarrow)$ be sound and terminating.*

- (a) *Every $r \in R$ rewrites to some normal form $\text{nf}(r)$ with $\llbracket \text{nf}(r) \rrbracket = \llbracket r \rrbracket$.*
- (b) *If moreover $\llbracket \cdot \rrbracket$ is injective on normal forms, then $\rightarrow$ is confluent, every r has a unique normal form, and for all $r, r' \in R$,*

$$\llbracket r \rrbracket = \llbracket r' \rrbracket \iff \text{nf}(r) = \text{nf}(r').$$

Proof. (a) By termination, any maximal chain from r is finite and ends in a normal form; by soundness the denotation is constant along the chain. (b) Two normal forms reachable from r both have denotation $\llbracket r \rrbracket$ by (a), so they are equal by injectivity; hence the normal form is unique. If $r \rightarrow s$ and $r \rightarrow t$, then $\text{nf}(s) = \text{nf}(r) = \text{nf}(t)$ (the normal form of s is reachable from r), so s and t join. For the equivalence, $(\Leftarrow)$ is soundness along both chains, and $(\Rightarrow)$ holds because $\text{nf}(r)$ and $\text{nf}(r')$ are normal forms with the same denotation. $\square$

Theorem 18.2(b) shows that confluence need not be proved as a separate lemma when termination, soundness, and injectivity on normal forms are available. The three properties together say that the representation is a presentation of the object and not a different object, and that a canonical representative can be reached by any strategy.

18.2 Integer radices

Proposition 18.3 (Integer radix). *Let $b \geq 2$ be an integer. Every natural number N has exactly one expansion $N = \sum_{k=0}^m d_k b^k$ with $d_k \in \{0, \dots, b-1\}$ and either $m = 0$ or $d_m \neq 0$.*

Proof. Existence by induction on N : for $N < b$ take $m = 0$; otherwise write $N = bq + r$ with $0 \leq r < b$ and $q < N$, expand q , and append r . Uniqueness: $d_0 \equiv N \pmod{b}$ is forced, so $d_0 = r$, and the remaining digits expand $(N - r)/b$, which has a unique expansion by induction. $\square$

Proposition 18.4 (Radix invariance). *Let $\nu_1 : S_1 \rightarrow \mathbb{N}$ and $\nu_2 : S_2 \rightarrow \mathbb{N}$ be bijections from sets of numerals, and define operations on numerals by transport, $x \oplus_i y := \nu_i^{-1}(\nu_i x + \nu_i y)$ and $x \otimes_i y := \nu_i^{-1}(\nu_i x \cdot \nu_i y)$. Then $\nu_2^{-1} \nu_1 : S_1 \rightarrow S_2$ is an isomorphism of $(S_1, \oplus_1, \otimes_1, \leq_1)$ onto $(S_2, \oplus_2, \otimes_2, \leq_2)$. Every sentence of first-order arithmetic is true in one if and only if it is true in the other.*

Proof. Both structures are isomorphic to $(\mathbb{N}, +, \times, \leq)$ by ν_1 and ν_2 respectively, and isomorphic structures satisfy the same first-order sentences. $\square$

Proposition 18.4 is a formal version of the statement that a number is not its decimal numeral: the choice of radix is a choice of ν , and every arithmetic fact is invariant under that choice.

18.3 A non-integer radix

The same schema applies to the radix $3/2$, which is a legitimate and non-degenerate choice of place-value system. The following rewriting presentation is the exploding-dot rule: N dots are placed in box 0, and three dots in box k may be replaced by two dots in box $k+1$.

Definition 18.5 (Unidimary system). Let R be the set of finitely supported sequences $c = (c_k)_{k \geq 0}$ of natural numbers (dots per box), with $\llbracket c \rrbracket := \sum_k c_k (3/2)^k \in \mathbb{Q}$. The rewrite $c \rightarrow c'$ holds when, for some k , $c_k \geq 3$, $c'_k = c_k - 3$, $c'_{k+1} = c_{k+1} + 2$, and $c'_j = c_j$ otherwise. The normal forms are the sequences with every $c_k \in \{0, 1, 2\}$, that is, the finite digit strings over $\{0, 1, 2\}$. The *unidimary numeral* of $N \in \mathbb{N}$ is the normal form reached from the configuration with N dots in box 0.

Lemma 18.6 (Soundness and termination). *The system of Definition 18.5 is sound and terminating. Starting from N dots in box 0, a normal form is reached after exactly $N - \sum_k d_k$ rewrites, where (d_k) is the final digit string.*

Proof. Soundness: $3(3/2)^k = 2(3/2)^{k+1}$, so a rewrite does not change $\llbracket \cdot \rrbracket$. Termination: each rewrite removes 3 dots and adds 2, so the total number of dots $\sum_k c_k$ decreases by exactly 1; it is a natural number, so only finitely many rewrites are possible. From N initial dots, reaching total $\sum_k d_k$ takes exactly $N - \sum_k d_k$ rewrites. $\square$

Lemma 18.7 (Injectivity on normal forms). *Two distinct finite digit strings over $\{0, 1, 2\}$, with leading zeros ignored, have distinct values $\sum_k d_k (3/2)^k$.*

Proof. Pad both strings to a common length $m + 1$ and suppose they have equal value. Let $e_k = d_k - d'_k \in \{-2, \dots, 2\}$ and suppose some $e_k \neq 0$; let j be the least such index. Multiplying $\sum_k e_k (3/2)^k = 0$ by 2^m gives $\sum_k e_k 3^k 2^{m-k} = 0$. Every term with $k > j$ is divisible by 3^{j+1} , and the terms with $k < j$ vanish. Dividing by 3^j yields $e_j 2^{m-j} + 3t = 0$ for an integer t . Hence $3 \mid e_j 2^{m-j}$, and since $\gcd(3, 2) = 1$, $3 \mid e_j$. But $0 < |e_j| \leq 2$, a contradiction. $\square$

Theorem 18.8 (Unidimary numerals). *Every natural number N has exactly one finite digit string over $\{0, 1, 2\}$ (without leading zeros) of value N . The exploding-dot rule reaches it from N dots in box 0 by every sequence of rewrites, in any order.*

Proof. By Lemma 18.6 a normal form is reached with value N , so a digit string of value N exists. Lemma 18.7 makes it unique. Any order of rewrites ends in a normal form of value N (Theorem 18.2(a)), which must be that string by uniqueness; this is Theorem 18.2(b). $\square$

Example 18.9. For $N = 10$: ten dots in box 0; three rewrites move six dots to box 1, leaving one in box 0; two rewrites in box 1 send four dots to box 2; one rewrite in box 2 sends two to box 3, leaving one. The final digits, from box 3 down, are 2, 1, 0, 1, and $2 \cdot \frac{27}{8} + 1 \cdot \frac{9}{4} + 0 + 1 = 10$. By Lemma 18.6 the number of rewrites is $10 - (2 + 1 + 0 + 1) = 6$, as counted.

Proposition 18.10 (Relation to rational-base numeration). *For a rational base p/q with $p > q \geq 1$ coprime, Akiyama, Frougny and Sakarovitch [1] represent each natural number M uniquely as $M = \sum_i a_i q^{-1} (p/q)^i$ with $a_i \in \{0, \dots, p-1\}$. For $p/q = 3/2$, the digit string (a_i) of M in their system is the unidimary numeral of $2M$.*

Proof. Multiplying the defining equation by $q = 2$ gives $\sum_i a_i (3/2)^i = 2M$ with $a_i \in \{0, 1, 2\}$. By Theorem 18.8, the only such digit string is the unidimary numeral of $2M$. $\square$

Proposition 18.10 fixes the attribution. The numeral systems of [1] are the literature on rational-base numeration. The unidimary presentation used here differs from it by the factor q^{-1} in the place values: it represents every natural number, including odd ones, and its numerals for even numbers $2M$ coincide with the numerals of M in [1]. The uniqueness

argument in Lemma 18.7 is elementary and is given here for completeness; the existence and uniqueness of the numerals in [1] is not used.

18.4 Why integer radices are used

Since Proposition 18.4 shows that every arithmetic fact is invariant, the reasons for preferring one radix are reasons of cost, and they can be measured.

Proposition 18.11 (Length lower bound). *A digit string of length m over $\{0, 1, 2\}$ has value at most $4((3/2)^m - 1)$. Hence the unidimary numeral of N has at least $\log_{3/2}(N/4 + 1)$ digits. For large N it is longer than the base- b numeral of N by a factor at least $\ln b / \ln(3/2)$ asymptotically; this is about 1.71 for $b = 2$ and 5.68 for $b = 10$.*

Proof. The value is at most $2 \sum_{k=0}^{m-1} (3/2)^k = 2 \cdot \frac{(3/2)^m - 1}{1/2} = 4((3/2)^m - 1)$. Solving for m gives the lower bound. The base- b numeral has $\lfloor \log_b N \rfloor + 1$ digits, and $\log_{3/2} N / \log_b N = \ln b / \ln(3/2)$. $\square$

Computation 18.12 (Lengths). Digits needed for N in base 10, base 2, and the unidimary system, with the bound of Proposition 18.11 rounded up.

N	base 10	base 2	unidimary	bound
10	2	4	4	4
100	3	7	9	9
1000	4	10	15	14
10^6	7	20	32	31

A second cost is redundancy of the digit alphabet. Over a base- b alphabet, every digit string is a numeral. Over $\{0, 1, 2\}$ in base $3/2$, most strings do not denote integers.

Computation 18.13 (Strings that are integers). Among the 3^l digit strings of length l (leading zeros allowed), the number whose value is an integer is:

l	1	2	3	4	5	6	7	8	9	10
integer-valued	3	6	9	15	24	36	54	81	123	186
3^l	3	9	27	81	243	729	2187	6561	19683	59049

The counts grow by a factor close to $3/2$ per digit, in line with the growth of the place values, so each additional digit adds well under one bit of capacity; a binary digit adds one. This is an observation about the table, not a theorem.

Analogy 18.14 (Arbitrary, but not equally good). The radices 2, 3, 10, and $3/2$ all give working numeral systems with identical arithmetic (Proposition 18.4). They differ in length, in the share of strings that are valid, and in how arithmetic algorithms behave. A convention of this kind is chosen on cost, and a shared choice is worth more than a locally optimal one. The same pattern holds one level down for foundations: the base is a convention, the consequences are determinate, and the reasons for adopting a particular base are reasons of cost and adoption.

Remark 18.15 (Four things that are not the same). The radix example separates four notions that are often run together. *Truth preservation*: every arithmetic sentence has the same truth value in every radix (Proposition 18.4). *Representational identity*: the

numerals differ, so the inscriptions are not the same. *Resource cost*: the lengths differ, with a proved lower bound (Proposition 18.11). *Practical interchangeability*: the systems are not interchangeable for use, because of cost and because most digit strings in the non-integer radix are not numerals (Computation 18.13). The same distinctions recur for foundations: ZFC and type theory may be related by interpretations (Cited result 13.1) without being identical, equally costly, or interchangeable in practice.

18.5 The representation schema

The radix example exhibits a four-step pattern.

Design rule 18.16 (Representation schema). A representation is described by (1) a denotation $\llbracket \cdot \rrbracket$ from inscriptions to the objects represented; (2) a set of admissible transformations of inscriptions; (3) an *invariant*, namely a preservation theorem that the denotation is unchanged by admissible transformations; and (4) a *canonical form*, a normal form reached by the transformations and identifying the denotation (Theorem 18.2).

Analogy 18.17 (The operators in the schema). Read through the schema, **Pop** generates candidate inscriptions, **Refuse** removes inadmissible ones (in Definition 18.5, the configurations not of the stated form are not inscriptions of the system, and, as Computation 18.13 shows, most digit strings do not denote integers), **Bind** ties inscriptions to their denotation, and **Collapse** selects or reaches the canonical representative. This is a reading of the schema, not a consequence of it.

The schema earns its generality only where the preservation theorem is proved. The table records where that is the case in this text.

Representation	Preservation result	Status
Unidimary numerals	Theorem 18.8	proved in this text
Boolean semantics	Soundness, Theorem 4.3	proved in this text
Terms as lambda terms	Proposition 3.2, Church–Rosser	proved by bijection of grammars, confluence cited
Typed terms to normal forms	Cited result 11.8	cited; fails for the untyped calculus, which does not terminate
Quotients	Proposition 11.15	proved from the stated kernel rules
Sets as pre-sets	Cited result 13.1	cited

Remark 18.18 (A criterion for failure). If, for a proposed use of the four operators, no denotation, no preservation theorem, and no canonical form can be exhibited, then the use is a renaming and the schema adds nothing to it. The untyped lambda calculus is the instructive boundary case in this document: its reduction is confluent but not terminating, so Theorem 18.2 does not apply to it, and canonical forms exist only for the terms that happen to normalize.

19 A reproducibility protocol

19.1 What a report must contain

The preceding sections suggest a minimal standard for reporting a formal result.

Design rule 19.1 (Reporting a formal result). A claim that a statement has been formally established should state:

- (1) the *base*: the theory or kernel and its version;
- (2) the *footprint* of the headline theorem (Definition 12.2);
- (3) the *layer* reached (Definition 15.1), and in particular that the kernel accepted the whole development;
- (4) which hypotheses are *declared as axioms by the author* rather than derived (Proposition 15.4);
- (5) the *scope*: a plain-language statement of what the theorem asserts, including what it does not;
- (6) the *environment*: pinned versions of the tools and libraries, so that the check can be repeated.

Items (1) to (3) are what turn “verified” into a claim that can be checked. Item (4) is the one most often omitted. A development in which the substantive assumptions enter as declared axioms is still a correct derivation, and by Proposition 15.4 what it establishes is the implication from those axioms. Reporting them separately lets a reader judge whether the axioms are the content of the claim.

19.2 Controls

Definition 19.2 (Controls). A *negative control* is a statement that is known to be false or ill-formed and that the pipeline must reject. A *positive control* is a statement known to be provable that the pipeline must accept.

Proposition 19.3 (A negative control detects trivial acceptance). *Suppose a checking pipeline accepts every derivation of $\perp$ from its base. Then it accepts every sentence φ with a one-line extension of that derivation. Hence a pipeline that rejects a single known-false sentence has not accepted a derivation of $\perp$.*

Proof. From a derivation d of $\perp$, adding the step $\perp \rightarrow \varphi$ (ex falso) and modus ponens gives a derivation of φ for any φ . Contrapositively, if some known-false sentence is rejected, then no derivation of $\perp$ was accepted, assuming the pipeline accepts all correct derivations from its base. $\square$

Controls test the pipeline, not the theorem. They are cheap and they exclude a class of silent failures, in particular a pipeline that accepts everything.

Layer	Failure to guard against	Control
Elaboration	Statement does not mean what it appears to mean	Include an ill-formed variant that must be rejected
Kernel	Checker accepts a bad term	Include a known-false statement that must be rejected
Footprint	Result silently depends on an extra axiom	Report the footprint of each headline theorem
Scope	Theorem proves less than its description claims	State the theorem in words beside the formal statement

19.3 What reproducibility does not give

- (i) It does not give *truth* about the subject matter. A result relative to T is a fact about derivability from T .
- (ii) It does not give *consistency*. By Gödel's second theorem (Cited result 9.1), a base strong enough for arithmetic cannot certify its own consistency; consistency is relative, as in Cited result 13.1.
- (iii) It does not give *interpretation*. A formalization can be accepted by the kernel and still fail to state what its authors say it states (layer (6) of Definition 15.1).

Analogy 19.4 (Common language). The three limits are the limits of any shared standard. A common unit does not tell one what to measure; a common character encoding does not tell one what to write; a common base does not tell one what is true. What each does is make it possible for a second party to repeat the first party's step and see the same outcome.

20 A reading of the RSVP plenum

20.1 Purpose and status

The author's RSVP framework describes a *plenum* $\Psi = (\Phi, \mathbf{v}, S)$ consisting of a scalar field Φ , a vector (transport) field $\mathbf{v}$, and an entropy field S , from which gravity-like effects are proposed to emerge as an entropic descent, with a compact form $\mathbf{G} = -\nabla S + \xi \nabla \Phi$. This section asks only whether the four operators supply a useful vocabulary for the plenum. *Nothing in this section derives the field equations of the plenum from the operators.* What is proved is one general fact about identification and information (Proposition 20.2); the remaining statements are labeled Analogy or Conjecture and carry no more warrant than that.

20.2 The operator reading

Analogy 20.1 (Plenum in four operators).

Operator	Reading in the plenum
Pop	a local distinction: a cell of the plenum, with a value of $(\Phi, \mathbf{v}, S)$
Refuse	the constraints that restrict which configurations count as admissible (boundary conditions, conservation constraints, admissible local sections)
Bind	coupling of neighbouring cells, in particular by the transport field $\mathbf{v}$
Collapse	coarse-graining: identifying microscopically different configurations that agree at a given scale

20.3 What is lost by identification

Collapse identifies items. Identification forgets which of the identified items was present. A quantitative form of this statement holds in any finite setting and is the only theorem needed.

Proposition 20.2 (Information lost by Collapse). *Let Ω be a finite world with a probability measure μ , let X be the random item with law μ , let $\sim$ be an equivalence relation on Ω and $q : \Omega \rightarrow \Omega/\sim$ the quotient map, and write H for Shannon entropy. Define the lost entropy $L(\sim) := H(X) - H(q(X))$. Then*

$$L(\sim) = H(X | q(X)) \geq 0,$$

with equality if and only if every $\sim$ -class contains at most one item of positive probability. Moreover, if $\sim_1$ refines $\sim_2$ then $L(\sim_1) \leq L(\sim_2)$, and

$$L(\sim_2) = L(\sim_1) + H(q_1(X) | q_2(X)).$$

Proof. Since $q(X)$ is a function of X , $H(X, q(X)) = H(X)$. The chain rule gives $H(X, q(X)) = H(q(X)) + H(X | q(X))$, hence $L(\sim) = H(X | q(X))$, which is nonnegative as a conditional entropy. It vanishes iff, given the class, X is determined almost surely, that is iff each class has at most one point of positive probability. For the refinement statement, $q_2 = r \circ q_1$ for a function r , so $H(X | q_2(X)) = H(X, q_1(X) | q_2(X)) = H(q_1(X) | q_2(X)) + H(X | q_1(X), q_2(X))$, and $H(X | q_1(X), q_2(X)) = H(X | q_1(X))$ because $q_2(X)$ is determined by $q_1(X)$. Nonnegativity of $H(q_1(X) | q_2(X))$ gives monotonicity. $\square$

Proposition 20.2 is an instance of the data-processing principle. Its role here is to give a precise, nonnegative, additive quantity attached to a Collapse: how much of the item's identity is not recoverable from its class. Coarser Collapse loses at least as much, and the loss composes additively along a chain of refinements.

20.4 An analogy and a conjecture

Analogy 20.3 (Entropy as the cost of Collapse). In the plenum, the entropy field S and the “entropic descent” of the framework have the same shape as Proposition 20.2: a coarse description discards information, the discarded amount is a nonnegative quantity that grows with coarseness, and systems under a coarsening flow move in the direction in which it grows. This resemblance suggests reading S as the lost entropy of a Collapse determined by the scale at which the plenum is described.

Conjecture 20.4 (Plenum entropy as lost entropy). *There is a family of equivalence relations $\{\sim_\lambda\}_\lambda$ on the configuration space of the plenum, indexed by a scale λ and refining as λ decreases, such that the pointwise lost entropy $S_\lambda(x) := L(\sim_\lambda)$ restricted to the cell at x agrees, in the continuum limit and up to normalization, with the entropy field S of the framework, and such that $-\nabla S_\lambda$ reproduces the entropic drift term of $\mathbf{G}$.*

Evidence. Only the structural resemblance of Analogy 20.3. The conjecture is heuristic. It is also easy to falsify in principle, because it requires an explicit choice of $\{\sim_\lambda\}$ and a comparison with the field equations of the framework.

20.5 Implications if the conjecture held

The implications listed are conditional, and each depends on Conjecture 20.4.

- (i) *No extra primitive.* The entropic term would be a property of the Collapse family and would not require a separate entropy primitive in the ontology.

- (ii) *Additivity across scales.* Proposition 20.2 gives S an additive decomposition along refinements, so contributions from successive coarse-grainings would add, which is a checkable constraint on any proposed form of S .
- (iii) *Foundations as Collapse families.* The revisions of Section 14 would be changes of which identifications are admitted. Strengthening the foundation by adding extensionality-type principles identifies more items; weakening it identifies fewer.

20.6 What would settle it

- (a) State a concrete family $\{\sim_\lambda\}$ on a lattice version of the plenum.
- (b) Compute S_λ for it and compare with the entropy equation of the framework in the continuum limit.
- (c) Show that the weak-field limit of $-\nabla S_\lambda$ yields the Poisson-type equation proposed in the framework, or identify precisely where it fails.

Each step is an open problem. A negative result at (b) or (c) would refute Conjecture 20.4 as stated while leaving the operator vocabulary of Analogy 20.1 intact.

21 Summary and open problems

21.1 What was built

1. A four-operator domain algebra (Section 2) with the laws of Refuse and the universal property of Collapse.
2. The untyped lambda calculus recovered by renaming, with Bind as abstraction and Collapse as evaluation (Section 3).
3. Classical and intuitionistic propositional logic as two disciplines of Refuse, with a proof of soundness and an explicit counterexample to excluded middle in the open-world discipline (Sections 4 and 5).
4. The axioms of ZFC as closure conditions, with proofs that the empty set is derived, that there is no universal set, that Foundation excludes membership loops, and that Choice implies excluded middle (Section 8).
5. The kernel of a Lean-style dependent type theory, with Refuse as subtype formation, Collapse as application and quotient, and the three axioms commonly added (Sections 11 and 12).
6. A comparison of sets and types in which the difference is located at the place where domain restriction is permitted (Section 13).
7. A taxonomy of the layers at which a formal statement may fail, and the relativity of any verified theorem to its axiom footprint (Section 15).
8. A reproducibility account in which the axioms of ZFC and a kernel's added axioms are conventions that supply a common language: reproducible results, composition over a common base, determinate consequences of an arbitrary base, and a reporting protocol with controls (Sections 16 to 19).

9. First-order logic with structures, satisfaction, the substitution and coincidence lemmas, soundness, and ZFC stated as a first-order theory (Section 7).
10. Simply typed lambda calculus with sums and the empty type, an explicit Curry–Howard theorem for a named calculus and a named proof system, and consistency of the object calculus by normalization (Section 10).
11. A worked derivation in the kernel, separating what is decided by computation ($2+2=4$, $n+0=n$) from what requires induction ($0+n=n$) (Section 11.6).
12. A diagonal schema and its four instances kept distinct (Section 9).
13. A radix case study that supplies a complete example of a representation with denotation, preservation, canonical form, and cost (Section 18).

21.2 Open problems

Conjecture 21.1 (Refuse-complete presentation). *There is a presentation of a Lean-style kernel in which the only binding construct is `Bind`, the only elimination is `Collapse`, and `Prop` and subtypes are obtained from a single restriction rule corresponding to `Refuse`, with the same provability strength as the presentation of Section 11.*

Status. Heuristic. The subtype construction of Definition 11.2 shows the restriction can be defined from the standard rules; the open question is the converse, whether the standard rules can be derived from a single restriction rule.

Further open problems:

1. Characterize the A-steps (Section 14) that are conservative over a given fragment, in the sense that they add no new theorems of a given form.
2. Give a mechanized development of Sections 4 to 11.7 in a proof assistant, so that every theorem labeled as proved here is machine-checked and carries a stated footprint.
3. Decide whether Conjecture 20.4 can be sharpened into a statement about a specific lattice model.

21.3 Exercises

Exercise 21.2. Show that $\text{Refuse}_P \text{Refuse}_Q = \text{Refuse}_Q \text{Refuse}_P$ for the domain-layer operator, and compute $\text{Refuse}_P \text{Refuse}_Q(\Omega)$ when $P \subseteq Q$.

Exercise 21.3. In the open-world semantics on $X = \mathbb{R}$, compute $\neg U$ and $\neg\neg U$ for $U = (0, \infty)$ and for $U = \mathbb{R} \setminus \{0\}$. Which of $\neg\neg U \subseteq U$ and $U \cup \neg U = X$ hold in each case?

Exercise 21.4. Reduce and $\top \top$ and plus $\bar{1} \bar{2}$ to normal form using only `Collapse- $\beta$` .

Exercise 21.5. Prove from Foundation that there is no infinite descending $\in$ -chain $x_0 \ni x_1 \ni x_2 \ni \dots$ when the chain is given as a set. (Hint: apply Foundation to $\{x_n : n \in \mathbb{N}\}$.)

Exercise 21.6. Write the term of type $\text{Refuse}_P(A) \rightarrow \text{False}$ when $P = \lambda x. \text{True}$ and check that it has the stated type under the rules of Section 11.

Exercise 21.7. Give a finite example $(\Omega, \mu, \sim)$ with $L(\sim) > 0$ and one with $L(\sim) = 0$ but $\sim$ not the identity relation. (Hint: use a class containing a point of probability zero.)

A Inventory of cited results

The table lists every result used without proof, with its source. All other numbered statements labeled Theorem, Proposition, or Lemma are proved in the text.

Item	Result	Source
4.4	Completeness of classical propositional logic by truth tables	[18]
5.2	Soundness and completeness of intuitionistic propositional logic for open-world semantics	[18]
8.7	Foundation is equivalent to every set having a rank	[13]
8.10	Independence of Choice from ZF	[11, 6]
8.12	Löwenheim–Skolem; V_κ models ZFC for inaccessible κ	[13, 14]
11.8	Normalization and consistency for the calculus of constructions with inductive types; relative consistency of the Lean kernel	[7, 8, 19, 5]
11.14	Function extensionality from <code>Quot.sound</code>	[5]
12.1	Diaconescu’s theorem in type-theoretic form	[9], library formalization
13.1	Sets in types and types in sets	[2, 3, 20, 5]
9.1	Gödel’s second incompleteness theorem	standard
7.10	Completeness and compactness of first-order logic	[23, 10]
10.5	Strong normalization and subject reduction for $\lambda^{\rightarrow \times + 0}$	[28, 22]
9.4	Fixed-point lemma; paradox for <code>Type : Type</code>	[21, 25]
18.10	Rational-base numeration of the natural numbers	[1]
16.2	Decidability of derivation checking for decidable axiomatized first-order theories	[10]
17.1	ZF and ZFC prove the same Π_2^1 sentences	[13]
17.2	Vitali set under AC; Solovay’s model	[17]

B Summary of status labels by part

Part	Content and warrant
Ground	Definitions; proved laws of Refuse and Collapse; renaming of the lambda calculus (proved by bijection of grammars)
Logic	Soundness proved; completeness cited; a proved counterexample to excluded middle in the open-world semantics
Sets	ZFC statements are standard; Propositions on $\emptyset$, universal set, Foundation, and Choice and quotients proved; independence and models cited
Types	Rules summarized from the literature; internal Refuse, negation and inhabitation, footprints proved; consistency and translations cited
Reflection	Revision, layers, and the reproducibility protocol are definitions with short proofs; conservativity and measurability are cited; the RSVP reading has one proved proposition and otherwise Analogy and Conjecture

C Script for the radix computations

The following script produces the unidimary numerals and the checks reported in Section 18. It was run, and its assertions held, for the ranges stated in the comments.

```

from fractions import Fraction as F
import itertools, math, random

def uni(n):
    # exploding-dot normal form, digits low to high
    b = [n]; i = 0
    while i < len(b):
        while b[i] >= 3:
            q = b[i] // 3
            b[i] -= 3 * q
            if i + 1 == len(b): b.append(0)
            b[i + 1] += 2 * q
        i += 1
    while len(b) > 1 and b[-1] == 0: b.pop()
    return ''.join(map(str, reversed(b)))

def val(s):
    return sum(int(c) * F(3, 2) ** k for k, c in enumerate(reversed(s)))

assert all(val(uni(n)) == n for n in range(3000)) # value conserved
for l in range(1, 10): # injectivity
    seen = {}
    for t in itertools.product('012', repeat=l):
        if t[0] == '0' and l > 1: continue
        v = val(''.join(t)); assert v not in seen; seen[v] = t
print([uni(n) for n in range(21)]) # table

```

The numerals for 0 to 20 are

0	0	6	210	12	2120	18	21200
1	1	7	211	13	2121	19	21201
2	2	8	212	14	2122	20	21202
3	20	9	2100	15	21010		
4	21	10	2101	16	21011		
5	22	11	2102	17	21012		

References

- [1] S. Akiyama, C. Frougny, and J. Sakarovitch. Powers of rationals modulo 1 and rational base number systems. *Israel J. Math.* 168 (2008), 53–91.
- [2] P. Aczel. The type theoretic interpretation of constructive set theory. In *Logic Colloquium '77*, North-Holland, 1978, 55–66.
- [3] P. Aczel. On relating type theories and set theories. In *Types for Proofs and Programs (TYPES '98)*, LNCS 1657, Springer, 1999, 1–18.
- [4] H. P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*. North-Holland, 1984.
- [5] M. Carneiro. The type theory of Lean. Master’s thesis, Carnegie Mellon University, 2019.
- [6] P. J. Cohen. The independence of the continuum hypothesis. *Proc. Natl. Acad. Sci. USA* 50 (1963), 1143–1148.

- [7] T. Coquand and G. Huet. The calculus of constructions. *Information and Computation* 76 (1988), 95–120.
- [8] T. Coquand and C. Paulin. Inductively defined types. In *COLOG-88*, LNCS 417, Springer, 1990, 50–66.
- [9] R. Diaconescu. Axiom of choice and complementation. *Proc. Amer. Math. Soc.* 51 (1975), 176–178.
- [10] H. B. Enderton. *A Mathematical Introduction to Logic*, 2nd ed. Harcourt/Academic Press, 2001.
- [11] K. Gödel. The consistency of the axiom of choice and of the generalized continuum hypothesis. *Proc. Natl. Acad. Sci. USA* 24 (1938), 556–557.
- [12] N. D. Goodman and J. Myhill. Choice implies excluded middle. *Z. Math. Logik Grundlagen Math.* 24 (1978), 461.
- [13] T. Jech. *Set Theory*, The Third Millennium Edition. Springer, 2003.
- [14] K. Kunen. *Set Theory: An Introduction to Independence Proofs*. North-Holland, 1980.
- [15] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE-28*, LNCS 12699, Springer, 2021, 625–635.
- [16] The mathlib Community. The Lean mathematical library. In *CPP 2020*, ACM, 2020, 367–381.
- [17] R. M. Solovay. A model of set-theory in which every set of reals is Lebesgue measurable. *Ann. of Math.* 92 (1970), 1–56.
- [18] A. S. Troelstra and D. van Dalen. *Constructivism in Mathematics*, Vol. I. North-Holland, 1988.
- [19] B. Werner. Une théorie des constructions inductives. PhD thesis, Université Paris 7, 1994.
- [20] B. Werner. Sets in types, types in sets. In *Theoretical Aspects of Computer Software (TACS '97)*, LNCS 1281, Springer, 1997.
- [21] G. S. Boolos, J. P. Burgess, and R. C. Jeffrey. *Computability and Logic*, 5th ed. Cambridge University Press, 2007.
- [22] J.-Y. Girard, Y. Lafont, and P. Taylor. *Proofs and Types*. Cambridge University Press, 1989.
- [23] K. Gödel. Die Vollständigkeit der Axiome des logischen Funktionenkalküls. *Monatsh. Math. Phys.* 37 (1930), 349–360.
- [24] W. A. Howard. The formulae-as-types notion of construction. In *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press, 1980, 479–490.
- [25] A. J. C. Hurkens. A simplification of Girard’s paradox. In *TLCA '95*, LNCS 902, Springer, 1995, 266–278.

- [26] F. W. Lawvere. Diagonal arguments and cartesian closed categories. In *Category Theory, Homology Theory and their Applications II*, LNM 92, Springer, 1969, 134–145.
- [27] P. Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, 1984.
- [28] W. W. Tait. Intensional interpretations of functionals of finite type I. *J. Symbolic Logic* 32 (1967), 198–212.